Executive Summary
Pembangunan Digital *Initiative Management System* (DIMS)

2026

Ditujukan untuk:

PT PLN (Persero)

Disusun oleh:

PT Akhdani Reka Solusi

Jl. Cikutra Baru No. 23 Bandung 40124

Telp/Fax (022) 7273691

contact@akhdani.co.id

www.akhdani.co.id

DAFTAR ISI

DAFTAR ISI 1

1 PENDAHULUAN 3

1.1 Latar Belakang 3

1.2 Maksud dan Tujuan 4

1.3 Ruang Lingkup 5

1.4 Batasan Ruang Lingkup 5

2 METODOLOGI PELAKSANAAN PEKERJAAN 6

2.1 Penerapan *Agile Methodology* 6

2.2 Penerapan Prinsip Manajemen Proyek 9

2.2.1 Manajemen Integrasi Proyek 10

2.2.2 Manajemen Lingkup Proyek 11

2.2.3 Manajemen Jadwal Proyek 12

2.2.4 Manajemen Pemangku Kepentingan (Stakeholder) Proyek 13

2.2.5 Manajemen Risiko Proyek 16

2.2.6 Manajemen Komunikasi Proyek 17

2.2.7 Manajemen Sumber Daya Proyek 19

2.2.8 Manajemen Kualitas Proyek 20

2.3 Konsideran Pengembangan 21

2.4 Asumsi 22

3 USULAN TEKNIS 23

3.1 *Platform* Teknologi 23

3.2 PERANCANGAN TOPOLOGI 23

3.3 Perancangan Arsitektur 25

3.4 Perancangan Data Model 27

3.5 Perancangan Antar Muka 28

3.6 Digital Workflow & Process Automation 29

3.6.1 *Workflow Engine* 29

3.6.2 *Business Process Repository* (Probis *Collection*) 32

3.7 Form Generator & Metada Management 34

3.8 Dashboard Monitoring & Report 36

3.9 File *Manager* 37

3.10 Logging & Audit Trail 37

3.11 Security, Governance, dan Audiatbility 38
3.11.1 *Implementasi Single Sign-On (SSO)* 38
3.11.2 Pengelolaan Hak Akses Pengguna 38
3.12 Strategi Pengujian 41
3.12.1 *Manual Testing* 41
3.12.2 *Automated Code Review* 41
3.12.3 *Manual Code Review* 43
3.12.4 *Automated Testing* 43
3.13 Strategi *Deployment* 45
3.13.1 *Version Control* 45
3.13.2 *Gitlab Runner* 47
3.13.3 Strategi *Deployment* 48
3.13.4 *Deployment and Operational* 49
4 Tahapan Pengembangan Aplikasi 49
4.1 Analisis Kebutuhan & Perancangan *Design* 49
4.2 Aplikasi (*Development*) 50
4.3 Review Hasil Pengembangan Aplikasi 52
4.4 Pengujian *User Acceptance Test* (UAT) 53
4.5 Deployment 54
4.6 Pemeliharaan dan Dukungan Teknis 54
4.6.1 Dukungan Teknis 55
4.6.2 Pendampingan Implementasi 55
5 ORGANISASI PROYEK BESERTA PERAN DAN TANGGUNG JAWAB TENAGA AHLI 56
5.1 Struktur Tim Proyek 56
5.2 Tugas dan Tanggung Jawab 57
6 TIMELINE 61
7 DELIVERABLE 63
8 PENUTUP 65

1 PENDAHULUAN

1.1 Latar Belakang

Dalam mendukung agenda transformasi digital perusahaan, PLN membutuhkan sistem yang tidak hanya mampu mendigitalisasi proses kerja yang berjalan saat ini, tetapi juga dapat beradaptasi terhadap kebutuhan bisnis yang terus berkembang. Seiring meningkatnya kompleksitas proses bisnis, jumlah pemangku kepentingan, serta kebutuhan pengendalian dan monitoring yang lebih baik, diperlukan sebuah *platform* yang mampu mengelola alur kerja secara terstandarisasi, transparan, dan terukur.

Saat ini, salah satu proses yang menghadapi tantangan tersebut adalah pengelolaan permohonan pengembangan aplikasi dalam bentuk *Change Request* (CR) dan J*ob Request* (JR) melalui Aplikasi Manajemen Surat (AMS). Proses pengajuan, verifikasi, pelaksanaan, hingga monitoring penyelesaian permohonan masih banyak dilakukan secara manual menggunakan excel dan media komunikasi terpisah. Kondisi ini menyebabkan keterbatasan visibilitas proses, tingginya potensi kesalahan administrasi, kesulitan monitoring SLA, serta belum tersedianya repositori informasi dan dokumen yang terpusat.

Selain itu, peningkatan jumlah permohonan dari berbagai divisi menunjukkan kebutuhan akan suatu mekanisme pengelolaan proses yang lebih terstruktur dan dapat dikembangkan secara berkelanjutan. Solusi yang dibangun tidak hanya perlu menjawab kebutuhan pengelolaan CR/JR saat ini, tetapi juga mampu mengakomodasi proses bisnis lainnya tanpa memerlukan pengembangan ulang yang signifikan.

Berdasarkan kebutuhan tersebut, solusi yang diusulkan adalah pembangunan **Digital *Initiative Management System* (DIMS)** yang merupakan sistem pengelolaan proses bisnis berbasis *workflow engine* dan *form generator* yang bersifat *configurable*. *Platform* ini dirancang untuk memungkinkan PLN mendefinisikan, mengelola, dan memonitor berbagai proses bisnis melalui konfigurasi tahapan, peran, aturan persetujuan, SLA, notifikasi, serta kebutuhan dokumen secara fleksibel. Kemampuan konfigurasi memungkinkan perubahan alur kerja, struktur persetujuan, maupun atribut proses dilakukan melalui sistem.

Sebagai implementasi tahap awal, *platform* ini akan digunakan untuk mendukung proses pengelolaan permohonan pengembangan aplikasi (CR/JR) yang saat ini menjadi kebutuhan prioritas. Seluruh tahapan mulai dari pengajuan permohonan, verifikasi, penilaian, persetujuan, pelaksanaan, monitoring progres, pengelolaan dokumen, hingga pelaporan akan dikelola dalam satu *platform* terintegrasi.

Dengan pendekatan tersebut, PLN memperoleh solusi yang tidak hanya menyelesaikan permasalahan operasional saat ini, tetapi juga membangun fondasi *platform* digital yang dapat dimanfaatkan secara berkelanjutan untuk mendukung berbagai kebutuhan otomasi dan pengelolaan proses bisnis di masa mendatang.

1.2 Maksud dan Tujuan

Pembangunan Aplikasi Digital *Initiative Management System* (DIMS) bertujuan untuk mendukung pengelolaan proses permohonan secara terpadu dari awal hingga akhir, mulai dari tahap pengajuan, pemantauan progres, hingga penyelesaian dalam satu *platform* yang terintegrasi. Adapun tujuan pengembangan aplikasi ini adalah sebagai berikut:

1. Menyediakan *platform* terintegrasi yang mampu mengelola proses permohonan secara *end-to-end*, mulai dari pengajuan, pemantauan, hingga penyelesaian permohonan. Saat ini, kapabilitas yang tersedia berfokus pada pengelolaan permohonan pengembangan aplikasi (CR/JR).
2. Menyediakan *platform workflow engine* yang mendukung pengelolaan, standarisasi, dan digitalisasi berbagai proses bisnis secara berkelanjutan, sehingga pengembangan proses baru dapat dilakukan lebih cepat, konsisten, dan adaptif terhadap perubahan kebutuhan organisasi.
3. Meningkatkan efisiensi dan evektivitas proses kerja melalui otomatisasi proses bisnis, sehingga dapat mengurangi aktivitas manual serta mempercepat proses pengambilan keputusan.
4. Memastikan transparansi dan keterbukaan informasi terkait status permohonan, sehingga seluruh stakeholder dapat memantau perkembangan setiap *Change Request* (CR) maupun *Job Request* (JR).
5. Mendukung pencapaian target *Service Level Agreement* (SLA) melalui mekanisme notifikasi otomatis kepada pihak yang bertanggung jawab, sehingga potensi keterlambatan penanganan dapat diminimalkan.
6. Menyediakan pusat repositori terintegrasi yang menyimpan seluruh data permohonan dan riwayat perubahan pada setiap tahapan proses untuk mendukung kebutuhan monitoring, pelacakan, dan audit secara akurat.
7. Mendukung percepatan transformasi digital perusahaan melalui penerapan proses pengelolaan pengembangan aplikasi yang lebih terstruktur, responsif, adaptif, dan sesuai dengan standar yang berlaku.
8. Meningkatkan sinergi dan koordinasi lintas fungsi terkait, misal antara BPO, MDG, STI, serta *stakeholder* lainnya, melalui penggunaan *platform* digital yang mendukung kolaborasi secara efektif.

1.3 Ruang Lingkup

Adapun ruang lingkup pengembangan aplikasi meliputi:

1. Pengembangan w*orkflow engine* untuk mengelola dan mengotomasi proses bisnis melalui konfigurasi alur kerja, tahapan, peran, SLA, persetujuan, notifikasi, penugasan, serta monitoring aktivitas dan histori perubahan secara terintegrasi.
2. Pengembangan *form generator* dan *metadata management* untuk mengelola form secara dinamis dan berbasis konfigurasi sehingga dapat digunakan pada berbagai proses bisnis.
3. Pemanfaatan *business process repository* (*probis collection*) sebagai master proses bisnis yang terintegrasi dengan *workflow engine* pada DIMS. Melalui pendekatan ini, *workflow* yang dibangun tidak disusun secara terpisah, melainkan mengacu pada proses bisnis, aturan bisnis, serta relasi entitas yang telah tersedia pada repository proses bisnis perusahaan.
4. *Use case* yang digunakan pada tahap ini adalah pengelolaan proses permohonan pengembangan aplikasi (CR dan JR) secara *end-to-end*, mulai dari pengajuan, verifikasi, persetujuan, monitoring progres, pelaporan realisasi, hingga evaluasi kinerja berdasarkan *Service Level Agreement* (SLA).
5. Pengembangan fitur master data, konfigurasi, keamanan, notifikasi, pengelolaan dokumen dan fitur pendukung lainnya untuk mendukung operasional aplikasi.
6. Integrasi dengan sistem eksisting PLN untuk mendukung pertukaran data dan *interoperabilitas* aplikasi.

1.4 Batasan Ruang Lingkup

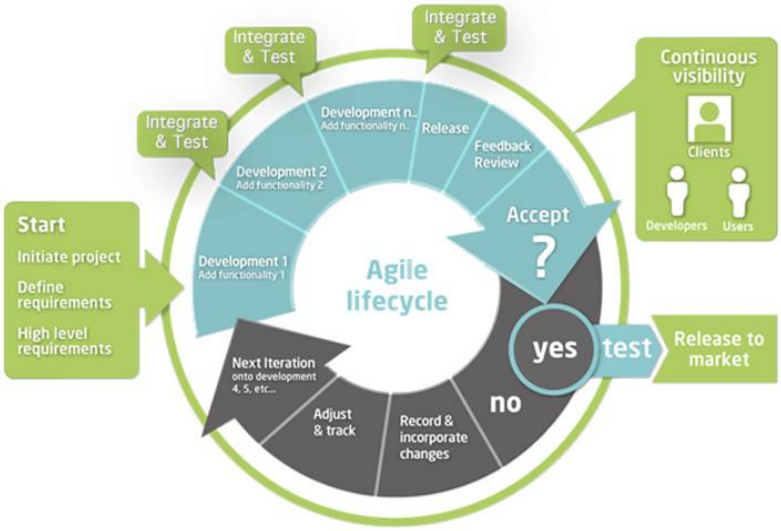
Adapun batasan ruang lingkup pengembangan aplikasi meliputi:

1. Pengembangan fokus pada *backbone* untuk *workflow engine* dan *form generator* & *metadata management*.
2. Sistem hanya menyediakan poses bisnis yang siap digunakan sesuai use case yang disepakati yaitu pengelolaan proses permohonan pengembangan aplikasi (CR dan JR) secara *end-to-end*, tanpa adanya fitur bagi pengguna untuk menyesuaikan urutan tahapannya di aplikasi.
3. Implementasi antarmuka difokuskan pada kesesuaian fungsionalitas dan konsistensi *style guide* yang telah disepakati bukan pada akurasi *pixel-perfect.*

2 METODOLOGI PELAKSANAAN PEKERJAAN

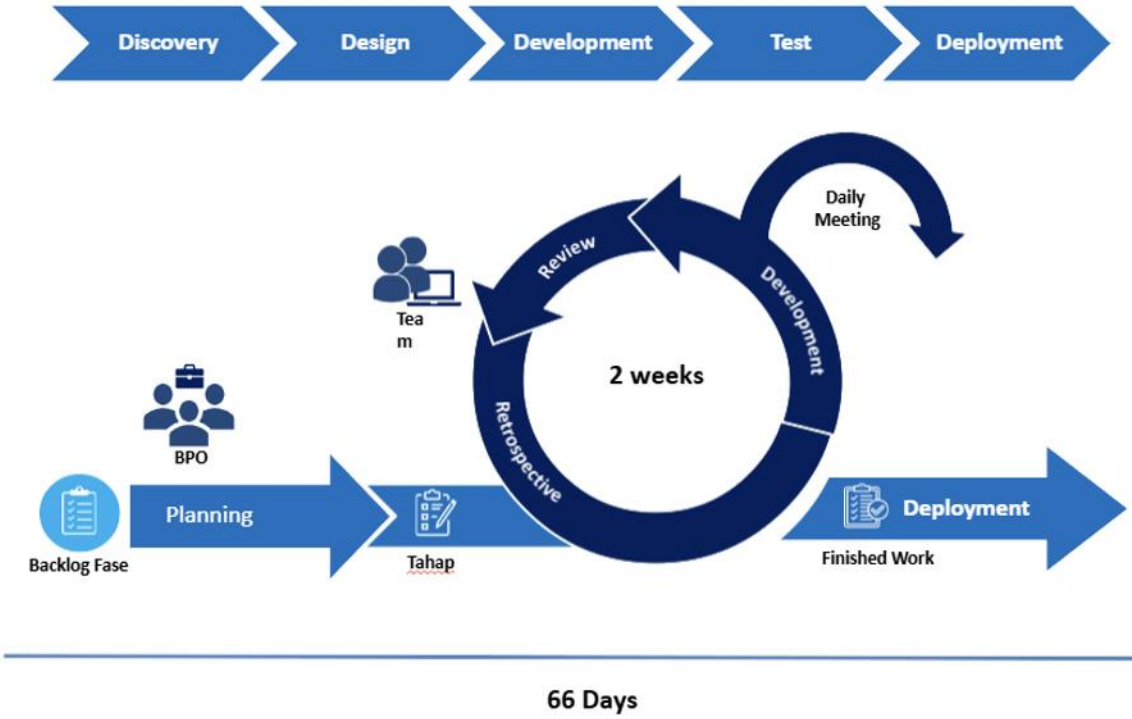
Tercapai atau tidaknya sasaran yang diharapkan dari pekerjaan ini, sangat tergantung pada metode pendekatan yang akan digunakan. Oleh karena itu, pemilihan metode pendekatan memiliki peran yang sangat strategis sebagai titik awal pelaksanaan pekerjaan dan akan menjadi acuan dalam mengelola seluruh tahapan kegiatan. Bagian ini menguraikan metode pendekatan yang digunakan tim penyedia barang atau jasa yang didasarkan atas pemahaman terhadap tujuan, sasaran, ruang lingkup dan hasil akhir pekerjaan yang diharapkan.

2.1 Penerapan *Agile Methodology*



Gambar 1 *Agile Methodology*

Agile Methodology adalah metodologi yang paling tepat untuk diterapkan. Kata *Agile* berarti cepat, ringan, fleksibel, dan adaptif, yang menggambarkan konsep model proses yang berbeda dari model-model tradisional yang sudah ada. Dalam *Agile software development*, proses pengembangan dilakukan secara iteratif yang lebih memfokuskan pada kolaborasi tim dan masukan langsung dari pengguna (*user feedback*) untuk penyempurnaan hasil di setiap iterasi. Tujuan penggunaan metodologi ini adalah mempercepat seluruh proses pengembangan dan implementasi, serta memastikan setiap hasil yang di-*deliver* selalu relevan dengan ruang lingkup kebutuhan bisnis PLN.



Gambar 2 Siklus Pengembangan

Siklus pengembangan sistem DIMS menggunakan pendekatan iteratif dengan periode waktu tetap (*time-boxed*) untuk menyelesaikan daftar pekerjaan (*backlog items*) yang telah disepakati. Proses *end-to-end* untuk menyelesaikan seluruh ruang lingkup pekerjaan ini ditargetkan berlangsung selama 66 hari kerja.

Dalam rentang waktu tersebut, pengerjaan tidak dilakukan secara linear, melainkan dibagi ke dalam beberapa siklus iterasi (*sprint*) dengan durasi 2 pekan (14 hari) untuk setiap tahapnya. Pendekatan iteratif per dua pekan ini secara khusus dirancang untuk mengedepankan prinsip ***release often*** **(rilis secara berkala) dan mengakomodasi** ***early feedback***. Dengan mendemonstrasikan atau merilis potongan fitur yang sudah berfungsi pada setiap akhir iterasi, tim pengembang dapat segera memvalidasi hasil kerja langsung dari para stakeholders.

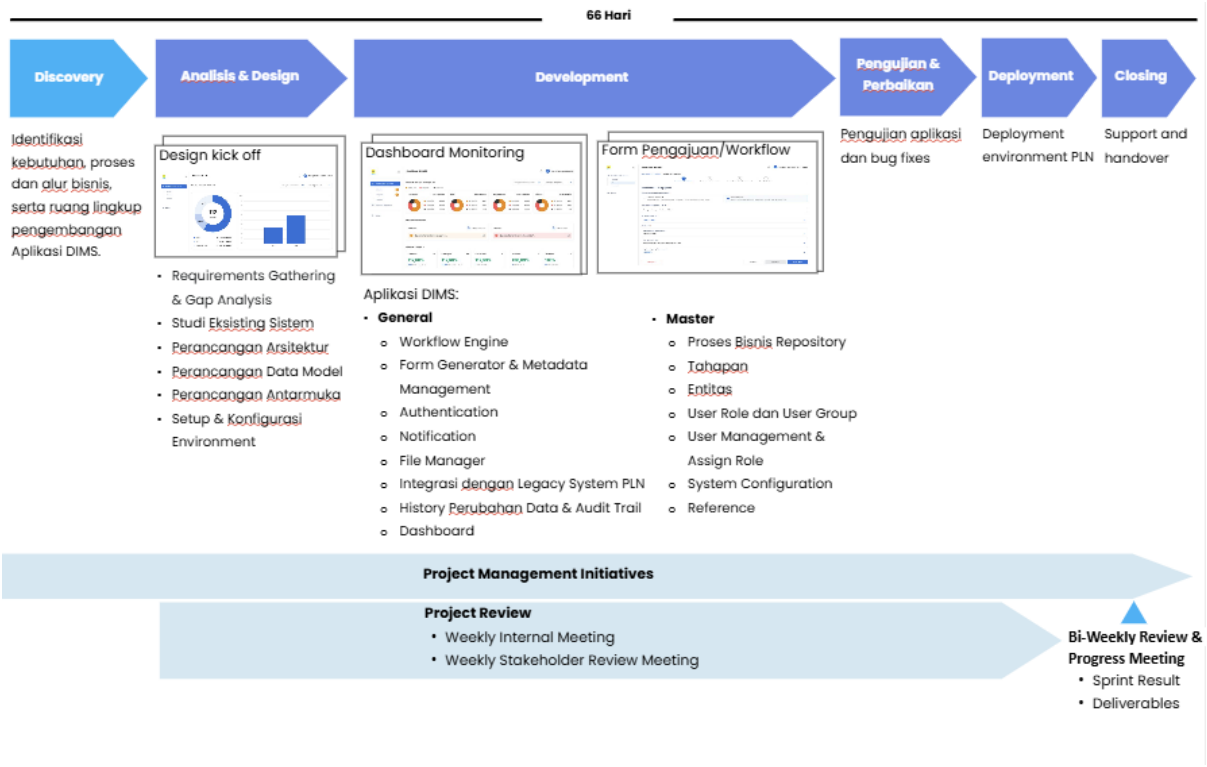
Secara garis besar, aktivitas *end-to-end* selama 66 hari tersebut mencakup tahapan berikut:

1. ***Planning*** dilakukan di awal pengembangan dan dihadiri oleh seluruh tim, yaitu BPO dan *Development Team*. Pada tahap ini, BPO menyampaikan prioritas pekerjaan, pain point, serta tujuan bisnis dari ruang lingkup pengembangan yang telah disepakati. Tim kemudian melakukan diskusi bersama untuk memahami kebutuhan, mengklarifikasi fitur yang akan dikembangkan,

serta menentukan prioritas item yang akan dimasukkan ke dalam *Backlog*. Hasil dari aktivitas ini adalah Rencana Kerja.

2. ***Sync Meeting*** adalah pertemuan singkat yang dilakukan setiap hari selama *sprint*. Pertemuan ini hanya dihadiri oleh *Development Team*. Pertemuan ini bertujuan untuk menemukan masalah lebih awal dan menyelaraskan kerja tim.
3. ***Development Work*** adalah inti dari tahapan proyek, berfokus pada pembuatan dan penyelesaian item atau fitur yang telah ditetapkan dalam *backlog* agar siap di-*deliver*. Dalam fase ini, tim *developer* melaksanakan serangkaian aktivitas teknis, meliputi implementasi fitur (*coding*), *unit* dan *integration test*, *code review* dan *refactoring*, *merging* ke *branch* utama, *build* dan *deployment* internal. Seluruh proses pengembangan dilakukan sesuai dengan *Definition of Done* (DOD), dan memastikan hasilnya berupa *deliverable* fungsional yang siap digunakan atau didemonstrasikan kepada *stakeholder*.
4. ***Bi-Weekly Review*** dan ***Bi-Weekly Progress Meeting*** merupakan pertemuan rutin yang dilaksanakan sesuai jadwal awal yang telah disepakati untuk meninjau kemajuan proyek. Pertemuan ini melibatkan seluru tim yaitu BPO, STI (*counter part*) dan *Development Team*. Fokus utama dalam pertemuan ini adalah pemeparan *progress* yang dicapai, yang diikuti oleh *demo progress* fitur yang sudah selesai. Diskusi kemudian berpusat pada review langsung dari BPO terhadap hasil pengerjaan, yang menjadi dasar untuk validasi *completed items* dan penetapan prioritas kerja untuk sprint berikutnya, memastikan proyek terus bergerak maju sesuai rencana dan kebutuhan pengguna.

Dengan menggunakan siklus pengembangan yang tetap, tim pengembang mampu memastikan bahwa setiap bagian dari pengembangan Aplikasi DIMS baik dari sisi teknis seperti data model dan integrasi, maupun dari sisi fungsional seperti menu kinerja dan aksi korporasi dapat disampaikan secara bertahap namun konsisten, sesuai *roadmap* implementasi fitur.

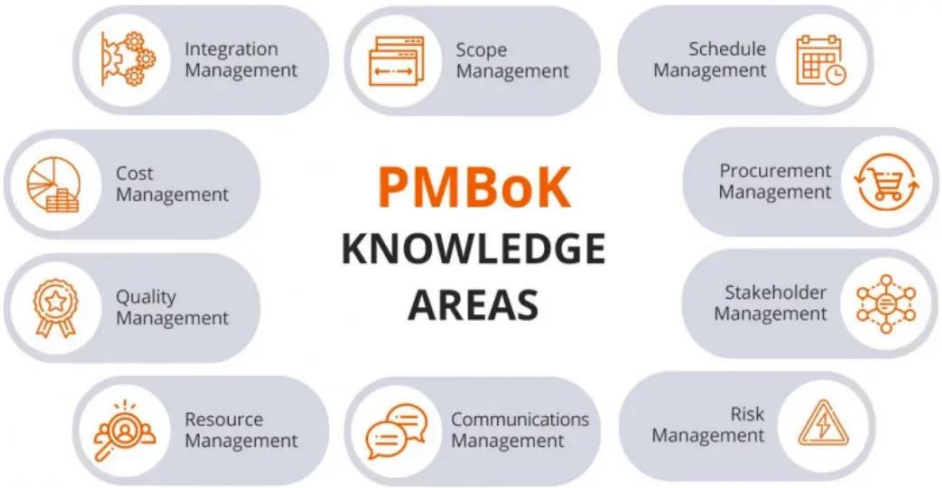


Gambar 3 *Best Practice* Pekerjaan Aplikasi DIMS

2.2 Penerapan Prinsip Manajemen Proyek

Manajemen proyek membantu menjaga proyek berjalan sesuai jadwal dan selesai tepat waktu. Manajer proyek mengkoordinasikan tugas dan dengan jelas mengidentifikasi tujuan atau hasil untuk mengurangi inefisiensi dalam manajemen waktu yang dapat mengakibatkan anggaran berlebihan dan pekerjaan tidak bisa selesai pada waktunya.

Untuk itu, manajer proyek harus menerapkan pengelolaan proyek dengan mengikuti *Framework* PMBOK *Guide*, 2017, p.23, yang digambarkan sebagai berikut:



Gambar 4 *Framework* PMBOK *Guide*, 2017, p.23

Berikut adalah area manajemen proyek yang akan diimplementasi dalam pekerjaan DIMS:

2.2.1 Manajemen Integrasi Proyek

Tujuan utama *Integration Management* adalah untuk mengelola proyek secara keseluruhan, dengan menyelaraskan aktivitas dan keputusan di berbagai *process groups*. Sebelum memulai pekerjaan, diperlukan koordinasi yang baik untuk memastikan tujuan dari project tercapai dan memenuhi semua ruang lingkup yang dibutuhkan. Koordinasi yang dilakukan seperti berkomunikasi aktif dengan *stakeholders*, merencanakan pekerjaan, mengalokasikan sumber daya, termasuk membuat keputusan atau solusi terbaik dengan mempertimbangkan batasan dan kebutuhan yang ada.



Gambar 5 *Integration Management*

Proses yang dilakukan pada tahap ini yaitu:

1. **Menyusun *Project Charter***

   Proses ini bertujuan untuk menyusun rencana keseluruhan yang akan digunakan untuk mengarahkan, melaksanakan, dan memantau proyek. Rencana ini mencakup berbagai aspek manajerial seperti jadwal, anggaran, manajemen risiko, dan sebagainya.

2. **Mengelola dan melaksanakan pekerjaan *project***

   Proses ini mencakup pelaksanaan pekerjaan yang telah direncanakan dalam rencana manajemen proyek, serta pengelolaan sumber daya, koordinasi aktivitas, dan pengawasan proyek agar tetap pada jalur yang benar.

3. **Monitoring dan mengendalikan pekerjaan *project***

   Proses ini berfokus pada pemantauan kinerja proyek terhadap rencana yang telah ditetapkan, identifikasi masalah atau perubahan yang diperlukan, dan melakukan tindakan korektif untuk memastikan proyek tetap sesuai dengan tujuan yang telah ditetapkan.

4. **Melakukan *Integrated Change Control***

   Proses ini melibatkan pengelolaan perubahan yang terjadi selama proyek, memastikan bahwa semua perubahan yang diperlukan disetujui melalui prosedur yang terstruktur, dan perubahan tersebut tercatat dengan baik untuk memastikan bahwa hasil akhir proyek tetap sesuai dengan tujuan.

5. **Melaksanakan proses penutupan proyek**

   Proses ini dilakukan untuk menyelesaikan seluruh kegiatan proyek atau fase proyek dengan cara formal, memastikan bahwa semua pekerjaan telah selesai, bahwa semua pihak yang terlibat puas, dan bahwa semua dokumentasi proyek telah diserahkan dan diselesaikan dengan baik.

Output dari integrasi proyek ini berupa sebuah dokumen *project charter* yang disetujui dan ditandatangani oleh semua pihak yang terlibat dan menjadi referensi selama proyek berlangsung.

2.2.2 Manajemen Lingkup Proyek

Manajemen lingkup proyek adalah aktivitas yang memastikan bahwa kegiatan suatu proyek adalah kegiatan yang diperlukan untuk menyelesaikan proyek dengan sukses. Salah satunya adalah mendefinisikan dan mengontrol apa yang termasuk ruang lingkup dan apa yang bukan. Berdasarkan PMBOK terdapat enam proses yang dilakukan saat aktivitas manajemen lingkup proyek:

1. ***Plan Scope Management***; perencanaan bagaimana ruang lingkup didefinisikan, divalidasi, diuji dan diawasi.
2. ***Collect Requirements;*** proses ini mengelola kebutuhan dan requirement dari para pemangku kepentingan (*stakeholder*) agar tetap selaras dengan tujuan proyek.
3. ***Define Scope;*** proses ini memfinalisasi hasil dari proses *collect requirements* yang akan dimasukkan ke dalam proyek, lalu membuat detail deskripsi detail, termasuk batasan dan *acceptance criteria*.
4. ***Create WBS;*** proses ini memecah-mecah *deliverable* dan aktivitas proyek menjadi komponen yang lebih kecil dan lebih mudah dikelola. Manfaat utamanya adalah menyediakan kerangka kerja apa yang harus di-*deliver* dari proyek ini. Proses ini umumnya dilakukan sekali di awal inisiasi proyek.
5. ***Validate Scope;*** proses ini memformulasikan kriteria *acceptance* dari *deliverable* proyek yang sudah selesai. Berguna untuk membuat proses *acceptance* lebih objektif dan meningkatkan kemungkinan acceptance dari pekerjaan dengan memvalidasi masing-masing *deliverable.*
6. ***Control Scope;*** proses ini bertujuan untuk memonitoring ruang lingkup proyek dan mengelola perubahannya. Setiap perubahan pada ruang lingkup harus melalui proses *change control* untuk mencegah terjadinya *scope creep* (penambahan ruang lingkup diluar kesepakatan) yang berpotensi membuat proyek terlambat secara jadwal dan meleset secara ruang lingkup.

Beberapa aktivitas yang dapat dilaksanakan terkait manajemen ruang lingkup adalah:

1. Pihak Akhdani dan PLN melakukan pembahasan bersama terkait latar belakang, *pain point*, dan ruang lingkup pekerjaan sehingga Akhdani bisa dengan tepat menentukan solusi yang diperlukan.
2. Melaksanakan proses *brainstorming* ide dengan BPO dan pihak terkait lainnya.
3. Menyusun rencana kerja menjadi beberapa tahapan, memilih dan menentukan *critical path* untuk setiap tahapan dan fitur yang akan dibangun.
4. Mensosialisasikan keseluruhan rencana dan ruang lingkup yang telah disusun kepada semua *stakeholder*.

2.2.3 Manajemen Jadwal Proyek

Manajemen jadwal proyek adalah proses untuk mengelola pelaksanaan proyek dari sisi waktu.



Gambar 6 *Project Scheduling Steps*

Setelah sumber daya, ruang lingkup, dan WBS telah disusun, maka akan dilakukan aktivitas berupa:

1. ***Plan Schedule Management;*** proses ini membuat aturan, prosedur, dan pendokumentasian atas perencanaan, eksekusi, dan pengawasan jadwal proyek.
2. ***Define Activities;*** proses ini mengidentifikasi dan mendokumentasikan aktivitas spesifik yang harus dilakukan untuk menghasilkan *deliverable* proyek.
3. ***Sequence Activities;*** proses ini menentukan hubungan dan ketergantungan di antara aktivitas proyek.
4. ***Estimate Activity Durations;*** mengestimasi durasi kerja yang diperlukan untuk menyelesaikan aktivitas dengan sumber daya yang tersedia.
5. ***Develop Schedule;*** proses menganalisis tahap-tahap di atas untuk membuat jadwal eksekusi dan monitoring proyek.

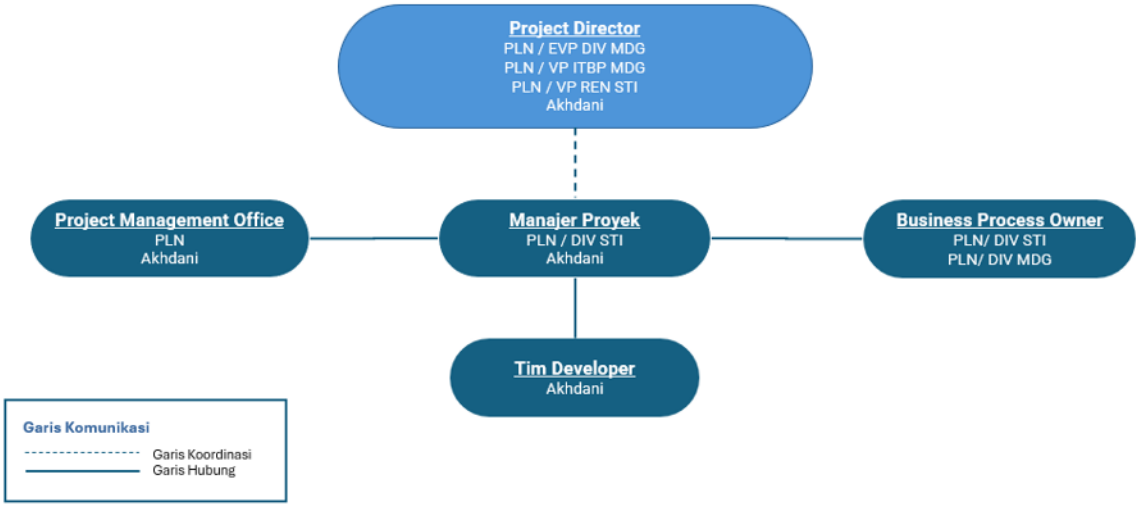
6. ***Control Schedule;*** proses mengawasi status proyek dan mengupdate jadwal dan mengelola perubahannya.

Berdasarkan pengalaman pada pekerjaan di lingkungan PLN sebelumnya, kegiatan yang biasa dilakukan pada aktivitas ini adalah:

1. Akhdani membuat rencana aktivitas dan jadwalnya pengerjaan dari ruang lingkup yang telah disepakati, dengan mempertimbangkan jumlah tenaga ahli yang mengerjakan, tingkat kesulitan, risiko, dan target selesai dari tim PLN.
2. Memaparkan hasil perencanaan jadwal dari Akhdani kepada tim PLN untuk mendapatkan persetujuan dan kesepakatan.
3. Jika ada hal yang belum sepakat maka rencana aktivitas akan direvisi untuk kemudian dipaparkan lagi hingga semua sepakat.
4. Jika di tengah eksekusi proyek ada kondisi yang mengharuskan perubahan jadwal, maka rencana aktivitas akan direvisi kemudian dipaparkan kembali untuk direview dan disetujui.

2.2.4 Manajemen Pemangku Kepentingan (Stakeholder) Proyek

Pemangku kepentingan atau *stakeholder* adalah pihak-pihak yang bisa mempengaruhi setiap proyek dari pekerjaan ini. Mengidentifikasi dan melibatkan mereka saat tahap perencanaan proyek sangat penting untuk memahami kepentingan, kebutuhan, dan kekhawatiran mereka atas proyek yang terlibat. Peran para *stakeholder* ini sangat esensial untuk mendukung keberhasilan proyek, dan keterlibatan mereka perlu dipertimbangkan selama masa pengerjaan proyek.

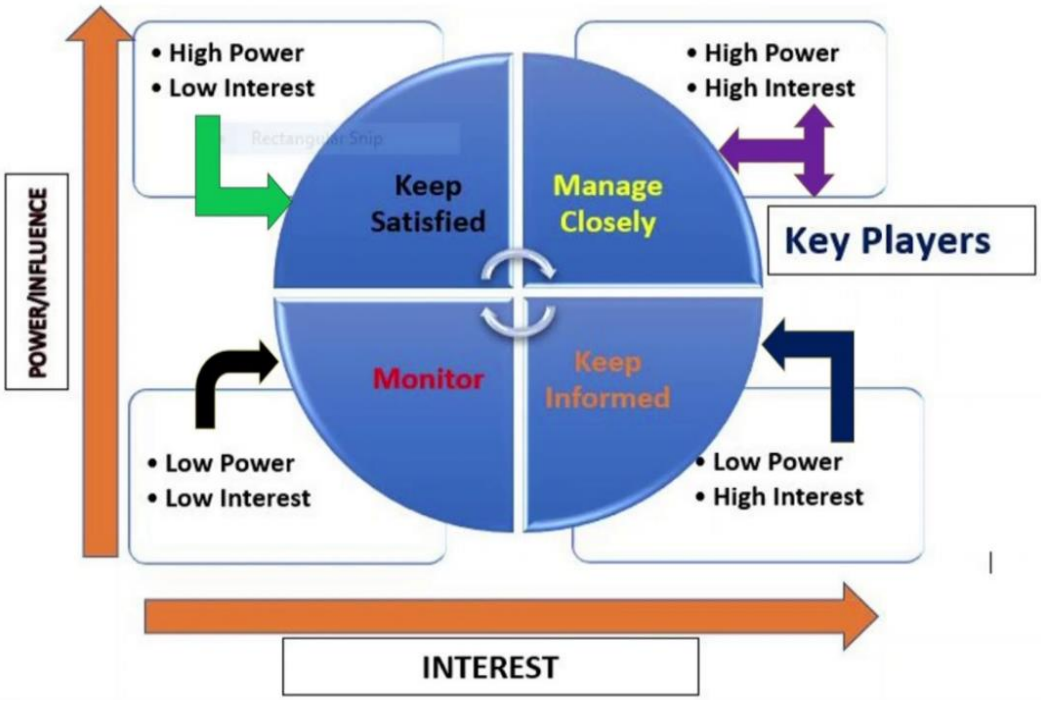


Gambar 7 Struktur *Stakeholder*

Stakeholder	Peran
DIV MDG	Owner dan Pengarah Strategi. Bertindak sebagai *Business Process Owner* (BPO), penentu spesifikasi strategis dan validasi, serta pemantauan pencapaian indikator.
DIV STI	*Counterpart* Teknologi, menampingi pengembangan aplikasi dari sisi teknis dan sistem. Pendukung utama dari sisi infrastruktur TI dan pengadaan proyek.
Pengguna Aplikasi di PLN *Holding*	Penerima hasil pengajuan CR JR, menginput *ideation*, dan mengikuti evaluasi melalui aplikasi. Melakukan *approval*, dan menggunakan aplikasi sebagai alat monitoring dan pengambilan keputusan.
Akhdani (Manajemen, Tim Teknis)	Bertanggung jawab atas desain, pengembangan dan pengujian aplikasi DIMS

Aktivitas yang terkait stakeholder adalah:

1. **Analisis *stakeholder***; aktivitas ini berisi proses identifikasi dan memahami berbagai macam individu ataupun kelompok yang memiliki andil dalam kesuksesan proyek. Dilakukan oleh Div MDG, bertujuan:
    - Memetakan siapa saja yang berkepentingan terhadap fitur, keamanan, integrasi data, dan antarmuka pengguna.
    - Menyesuaikan kebutuhan informasi dan laporan setiap dashboard pada stream BPO, stream STI, stream MDG dan stream PKK.
2. **Penentuan prioritas**; aktivitas ini adalah proses menentukan prioritas *stakeholder* mana yang paling penting untuk difokuskan keterlibatannya. Pada tahap ini kita bisa menggunakan *power/interest grid* untuk menentukan apa yang harus dilakukan pada masing-masing *stakeholder*.



Gambar 8 *Stakeholder Management*

Grid ini terdiri dari empat kuadran, yaitu.

a. *High power - high interest* (terlibat penuh)

   Stakeholder yang ditempatkan di bagian ini adalah yang utama. Perlu menjalin komunikasi sesering mungkin dengan mereka. Contoh *stakeholder* ini adalah pimpinan dari user aplikasi yang sedang dikembangkan pada kontrak rinci yaitu DIV MDG.

b. *High power - low interest* (tetap puas)

   Untuk menjalin hubungan dengan bagian ini, harus melakukan cukup upaya agar membuat mereka merasa puas, misalnya memberikan informasi saat progress pekerjaan mencapai suatu *milestone*. Contoh *stakeholder* ini adalah pimpinan divisi STI di PLN.

c. *Low power – high interest* (tetap mendapatkan informasi)

   Stakeholder yang masuk pada bagian ini tetap perlu diberikan informasi memadai tentang proyek. Memastikan bahwa mereka tidak memiliki masalah dengan proyek tersebut dan tetap berjalan sesuai tujuan awal. Contoh *stakeholder* ini adalah staf-staf *non-user* aplikasi.

d. *Low power - low interest* (usaha minimum)

   Kelompok ini cukup diberikan informasi memadai tanpa membebani mereka. Contohnya staf divisi lain yang sedikit bersinggungan dengan divisi yang terlibat di proyek.

3. Menjalin komunikasi dengan *stakeholder*; setelah memahami siapa saja dan bagaimana prioritasnya, selanjutnya aktivitas ini akan mulai menjalin komunikasi dengan para stakeholder berdasarkan prioritasnya. Mulai dari inisiasi proyek saat rapat *kick-off*, rapat rutin untuk update progress proyek, rapat membahas risiko dan kendala, hingga saat tahap penyelesaian proyek.

2.2.5 Manajemen Risiko Proyek

Proses *risk management* tidak bisa dipisahkan dari Pengembangan Aplikasi, mengingat PLN yang bergerak di industri yang *highly regulated*. *Risk assessment* perlu dilakukan pada modul-modul aplikasi sebagai dasar dari mitigasi risiko.



Gambar 9 *Risk Management* saiga dasar dari mitigasi risiko

Langkah-langkah yang dilakukan adalah sebagai berikut:

1. Melakukan inisialisasi *Risk Assessment* dan menentukan *system impact*
2. Melakukan identifikasi fungsi sistem yang memiliki dampak terhadap keselamatan / *safety*, kualitas produk / layanan dan integritas data
3. Melakukan *functional risk assessment* terhadap modul dan fitur aplikasi
4. Mengimplementasikan metode dan langkah pengendalian sesuai hasil *risk class* dan *risk priority*
5. Melakukan review risiko dan monitoring

Langkah-langkah lebih lanjut terkait *functional risk assessment* terhadap modul, yaitu:

1. Untuk fitur-fitur sesuai pada dokumen user *requirement* dilakukan *risk-assessment* untuk menentukan risk class berdasarkan *impact / severity* dan *probability*
2. Selanjutnya ditentukan r*isk priority* berdasarkan *risk class* dan *detectability*

3. Untuk modul dan fungsi yang memiliki *risk priority* tinggi akan dilakukan skenario pengujian yang lebih intensif sebagai salah satu bentuk mitigasi risiko.



Gambar 10 Metode *Functional Risk Assessment*

2.2.6 Manajemen Komunikasi Proyek



Gambar - *Steps in Project Communication Management*

Berdasarkan PMBOK, proses manajemen komunikasi proyek adalah proses untuk memastikan kebutuhan informasi proyek dan para pemangku kepentingan terpenuhi melalui artefak dokumen ataupun aktivitas yang didesain untuk mencapai pertukaran informasi yang efektif.

Manajemen komunikasi proyek ini terdiri dari dua bagian, yaitu: a) mengembangkan strategi komunikasi, dan b) pelaksanaan aktivitas untuk mengimplementasikan strategi yang telah dibangun. Proses yang ada pada manajemen komunikasi adalah:

1. ***Plan Communications Management;*** proses membuat rencana untuk aktivitas komunikasi yang akan dilakukan berdasarkan kebutuhan informasi masing-masing *stakeholder*, kelompok, serta berdasarkan aset organisasi yang tersedia dan kebutuhan proyek.
2. ***Manage Communications;*** proses untuk memastikan bahwa waktu dan aliran informasi yang beredar baik keluar atau masuk, dari satu pihak ke pihak lain sudah tepat dan benar.

3. ***Monitor Communications;*** proses monitoring untuk memastikan kebutuhan informasi proyek dan para pemangku kepentingan sudah terpenuhi.

Pada saat akan dimulainya pekerjaan, akan ditentukan dan disepakati metode komunikasi dan kolaborasi yang akan dilakukan sebagai sarana untuk memberi informasi, meminta konfirmasi, ataupun meminta jawaban kepada tim PLN terkait suatu kondisi pekerjaan. Aktivitas yang umum disepakati adalah sebagai berikut:

1. Komunikasi Interaktif

   Komunikasi interaktif adalah bentuk komunikasi dimana semua orang yang terlibat langsung saling berbicara, salah satu contoh aktivitasnya adalah rapat. Rapat adalah komunikasi formal yang bertujuan membahas suatu agenda, menarik kesimpulan, dan menentukan aksi lanjutannya (*next action*). Rapat akan dilakukan antara Akhdani, *counterpart* PLN, dan tim lain yang berkepentingan untuk berdiskusi, melaporkan progress, dan lain sebagainya.

   Setiap pekerjaan akan memiliki perbedaan kebutuhan komunikasi yang akan disepakati saat dimulainya pekerjaan. Aktivitas rapat yang umumnya dilakukan adalah:

   a. Rapat yang tidak rutin seperti *Kick Off Meeting,* UAT, *Training*, dan lain sebagainya.
   b. Rapat rutin mingguan (*weekly*) atau setiap dua minggu (*biweekly*), umumnya digunakan untuk laporan kemajuan pekerjaan (*progress report*).
   c. *Ad-Hoc Meeting*, meeting yang biasa digunakan untuk membahas isu yang membutuhkan pembahasan dan solusi penanganan cepat.

   Rapat dapat dilakukan secara *offline*, *online*, atau *hybrid* sesuai kebutuhan dan urgensi. Untuk *tools* rapat daring beberapa yang direkomendasikan adalah Zoom Meeting, Google Meet, dan Microsoft Teams.

   Semua hasil rapat akan didokumentasikan dan hasil diskusi akan dituangkan dalam *Minutes of Meeting* (MoM) sebagai bukti bahwa telah terjadi rapat dan semua pihak menyetujui hasil rapat yang ada dalam MoM tersebut.

2. Komunikasi Selain Rapat

   Selain perihal rapat, di awal pekerjaan juga akan ditentukan jalur komunikasi melalui pesan teks seperti email atau aplikasi pesan instan WhatsApp. Jalur ini paling bermanfaat untuk menginformasikan hal-hal yang ukurannya sangat kecil misal informasi ketersediaan jadwal peserta rapat atau pertanyaan teknis yang sederhana, sehingga tidak perlu menghabiskan

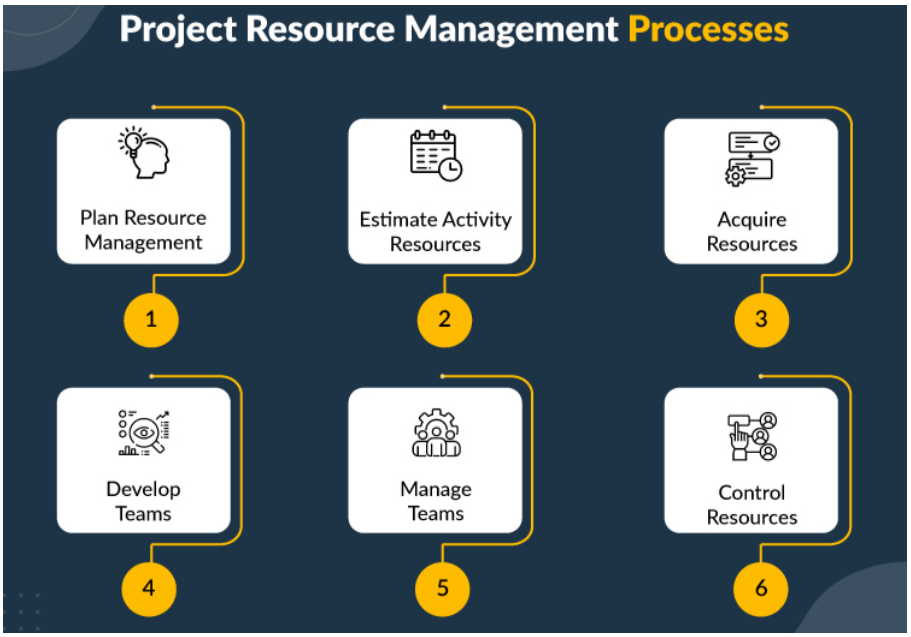
waktu untuk bergabung dalam ruang rapat untuk hal yang bisa dikomunikasikan via teks. Namun jika jalur teks ini mulai tidak produktif maka akan diarahkan ke *ad-hoc meeting*.

3. Kolaborasi Lain

   Kolaborasi lainnya yang tidak kalah penting pada pekerjaan ini adalah kolaborasi yang bertujuan untuk berkomunikasi dengan seluruh anggota tim dan berbagi dokumen. Hal ini dilakukan untuk meningkatkan produktivitas dan efisiensi. Beberapa tools yang bisa jadi pertimbangan di antaranya:

   a. Microsoft Teams, digunakan untuk berbagi informasi dalam bentuk *channel/room chat* yang mudah diakses.
   b. Draw.io, digunakan untuk membuat diagram/ilustrasi, bisa juga digunakan saat rapat.
   c. Google Drive atau One Drive, digunakan untuk berbagi dokumen.

2.2.7 Manajemen Sumber Daya Proyek



Gambar - *Project Resource Management Processes*

Manajemen sumber daya merupakan proses mengidentifikasi, mendapatkan, dan mengelola sumber daya tenaga ahli yang diperlukan untuk pelaksanaan proyek. Proses ini memastikan tenaga ahli yang tepat tersedia bagi tim proyek tersebut, memetakan tenaga ahli sesuai dengan kebutuhan, mengatur alur kerja sampai memantau performa dari tenaga ahli menjadi penting pada suatu proyek.

Berdasarkan PMBOK, proses pada tahapan ini adalah:

1. Perencanaan dan estimasi aktivitas sumber daya; berdasarkan kebutuhan lingkup pekerjaan dan sumber daya yang telah ditentukan pada kontrak rinci, akan dilakukan penentuan personil

yang ditugaskan untuk pekerjaan tersebut. Hal ini dilakukan mengikuti kesesuaian peran dan ketersediaan personil pada saat itu serta dapat dilakukan pada saat *sprint planning*.

2. Mengumpulkan, mengembangkan dan mengelola tim; mengelola dinamika tim sehari-hari, mengelola komunikasi tim dan hal-hal yang diperlukan untuk memastikan kinerja yang optimal. Hal ini dapat dilakukan pada saat *daily Meeting*
3. Monitoring and *controlling*; memastikan penggunaan sumber daya sesuai dengan rencana dan melakukan penyesuaian jika diperlukan.

Pada pekerjaan ini, sumber daya tenaga ahli yang diajukan adalah sebagai berikut:

1. *Project Manager*
2. Analis Bisnis
3. Analis Sistem
4. Developer Senior
5. Developer Junior
6. DevOps
7. *Quality Assurance*
8. *Technical Writer*

Untuk detail dari pengelolaan SDM pada pekerjaan ini akan dijelaskan pada Bab 4 - Organisasi Proyek Beserta Peran dan Tanggung Jawab Tenaga Ahli.

2.2.8 Manajemen Kualitas Proyek

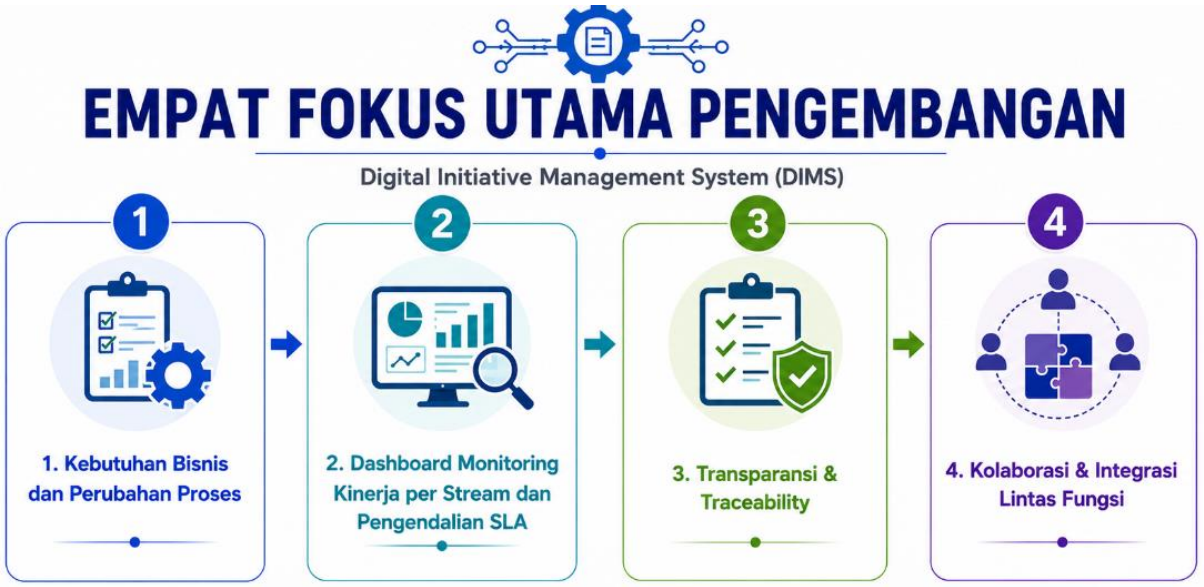
Menjaga kualitas hasil pekerjaan proyek aplikasi dapat dicapai dengan melakukan pengujian pada aplikasi dengan memastikan hasil pekerjaan sudah memenuhi standar yang disyaratkan dan hasilnya memuaskan. Proses pengujian pada pekerjaan ini akan dilakukan secara konkuren dengan proses *development* sehingga lebih cepat mengidentifikasi dan menyelesaikan *defect*. Proses ini juga memiliki *benefit* berupa tiap *developer* mendapatkan *feedback* lebih cepat dan *project manager* mendapat *visibility* lebih baik.

Proses pelaksanaan pengujian aplikasi masih dilakukan secara manual. Pengujian berbasis skenario yang dilakukan oleh tenaga ahli *Quality Assurance* secara langsung pada aplikasi yang sedang dikembangkan. Contohnya pada aplikasi berbasis web, tenaga ahli QA dapat langsung menggunakan web browser untuk mengakses dan menguji aplikasi.

Hasil pengujian dapat dituangkan ke dalam dokumen hasil pengujian yang disebut sebagai *User Acceptance Test*. Untuk setiap *defect* yang tidak memenuhi standar, tenaga ahli QA kemudian meneruskan ke tenaga ahli developer untuk diperbaiki.

2.3 Konsideran Pengembangan

Konsideran pengembangan terkait usulan inovasi yang Akhdani ajukan dilandasi dari rencana pengembangan DIMS yang tercantum pada BRD maupun kondisi lapangan yang telah diketahui. Beberapa hal yang menjadi konsideran antara lain sebagai berikut :



Gambar - Konsideran Pengembangan

1. **Kebutuhan bisnis dan perubahan proses**

   Perkembangan kebutuhan bisnis PLN serta perubahan proses dan kebijakan yang berlaku mendorong perlunya pengembangan dan penyesuaian aplikasi untuk meningkatkan efisiensi operasional, mendukung transformasi digital, dan memastikan keselarasan dengan kebutuhan perusahaan.

2. **Dashboard Monitoring Kinerja per Stream dan Pengendalian SLA**

   Meningkatnya jumlah inisiatif dan aktivitas pada berbagai stream kerja di PLN mendorong kebutuhan akan dashboard monitoring yang mampu memberikan visibilitas kinerja, memantau pencapaian SLA, serta mendukung pengambilan keputusan secara cepat dan tepat.

3. **Transparansi & Traceability**

   Pengelolaan permohonan pengembangan aplikasi melibatkan berbagai pihak dan tahapan proses yang memerlukan *traceability* yang jelas. Oleh karena itu, diperlukan sistem yang dapat merekam setiap aktivitas, perubahan status, dan keputusan yang diambil selama siklus permohonan berlangsung. Ketersediaan informasi historis tersebut mendukung transparansi

proses, memperkuat akuntabilitas, serta memudahkan pelaksanaan monitoring, evaluasi, dan audit.

4. **Kolaborasi & Integrasi Lintas Fungsi**

   Keterlibatan berbagai divisi dalam pelaksanaan inisiatif digital di PLN mendorong kebutuhan akan sistem terintegrasi yang mendukung kolaborasi, koordinasi, dan pertukaran informasi secara efektif guna meningkatkan efisiensi serta memastikan keselarasan pelaksanaan kegiatan antar divisi.

2.4 Asumsi

1. BRD digunakan sebagai acuan kebutuhan *use case* awal (CR/JR), sedangkan kapabilitas dan desain *platform* DIMS disusun berdasarkan hasil *requirement gathering* bersama Divisi MDG.
2. *Business process repository* pada tahap implementasi saat ini bersifat *read only*, dimana data proses bisnis dikelola dan disimpan pada data layer yang telah ditentukan.
3. *Form generator* difokuskan untuk memenuhi kebutuhan konfigurasi dan pengelolaan form secara dinamis. Tampilan form yang dihasilkan tidak ditujukan untuk memenuhi kebutuhan desain antarmuka yang bersifat *pixel perfect*.
4. Integrasi dengan sistem *legacy* PLN dilakukan sesuai ketersediaan layanan integrasi, spesifikasi teknis, akses lingkungan pengujian, serta dukungan dari pemilik sistem terkait.
5. Mekanisme autentikasi dan otorisasi pengguna memanfaatkan layanan identitas yang tersedia di lingkungan PLN atau disesuaikan dengan kebijakan keamanan yang berlaku.
6. *Infrastruktur server*, jaringan, domain, sertifikat, akses lingkungan *deployment*, dan kebutuhan pendukung lainnya disediakan oleh PLN sesuai jadwal implementasi yang telah disepakati.
7. Data referensi, master data, struktur organisasi, role pengguna, serta kebutuhan konfigurasi proses bisnis disediakan dan divalidasi oleh pihak PLN selama pelaksanaan proyek.
8. Pelaksanaan *User Acceptance Test* (UAT) dilakukan oleh pengguna yang ditunjuk oleh PLN, sedangkan Akhdani memberikan dukungan perbaikan terhadap temuan yang masih berada dalam ruang lingkup pekerjaan.
9. Dokumen teknis, konfigurasi, dan hasil pengujian yang menjadi *deliverable* disusun berdasarkan fitur dan ruang lingkup yang diimplementasikan pada proyek ini.

3 USULAN TEKNIS

3.1 *Platform* Teknologi

Berikut ini merupakan usulan *platform* teknologi yang kami ajukan untuk mendukung proses Pengembangan DIMS di PLN.

No.	Kategori	Teknologi	Notes
1.	*Front End & Form Builder*	Vue.js	Vue.js 3.5.*
2.	*Backend & Workflow Engine*	GO	Golang
3.	*Storage*	MINIO	MinIO 9.0.*
4.	*Database*	PostgreSQL mongoDB.	Relational Database: PostgreSQL 16.* Document Type Database: MongoDB 8.3.*
5.	CI/CD	GitLab SonarQube	Mengikuti infrastruktur di PLN

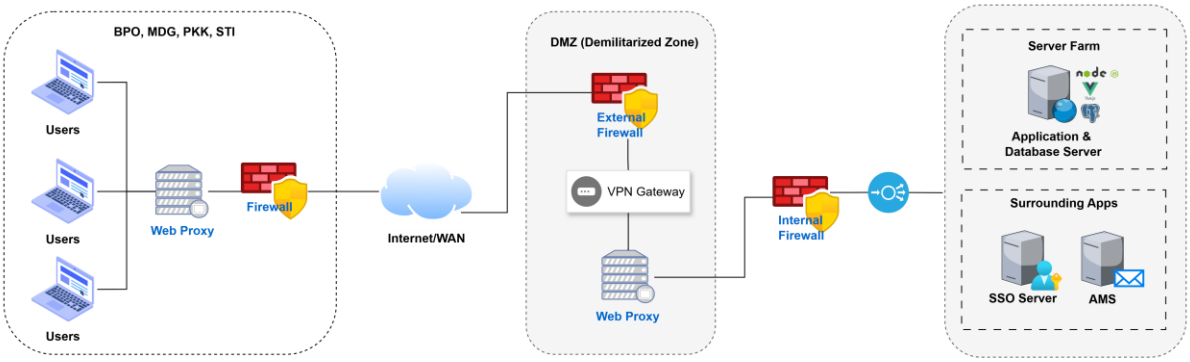
Pemilihan *platform* teknologi ini dilakukan secara cermat dengan mempertimbangkan kebutuhan fungsional aplikasi, skalabilitas sistem, kemudahan pemeliharaan, kompatibilitas dengan ekosistem teknologi PLN, serta tingkat keamanan dan keandalannya. *Platform* yang diusulkan terdiri dari kombinasi perangkat lunak, bahasa pemrograman, *framework*, serta arsitektur sistem yang dirancang untuk memastikan aplikasi dapat berjalan secara optimal dalam jangka panjang.

Usulan *platform* ini juga bertujuan untuk mendukung pengembangan aplikasi yang responsif, interaktif, dan fleksibel, khususnya dalam memastikan keberhasilan pelaksanaan rencana jangka panjang, termasuk inisiatif strategis yang telah diuraikan dalam peta jalan organisasi. Dengan memanfaatkan teknologi seperti visualisasi data dan analitik otomatis, sehingga akan menghasilkan kesenjangan antara perencanaan strategis dan pelaksanaan memastikan perbaikan berkelanjutan dengan hasil yang lebih baik.

3.2 PERANCANGAN TOPOLOGI

Perancangan topologi ini dilakukan untuk menjamin keamanan akses aplikasi dan memastikan efisiensi komunikasi data antara pengguna, aplikasi, dan sistem pendukung lainnya. Perancangan ini

bertujuan untuk memastikan Aplikasi DIMS dapat diakses dengan aman di internal PLN. Dalam konteks pengembangan ini, topologi dirancang untuk menyediakan arsitektur yang aman dan *scalable*, mendukung akses terbatas dan terkontrol dari berbagai unit PLN melalui *web proxy* dan *firewall*, memastikan aplikasi dari jaringan luar dengan tetap memungkinkan akses aman melalui VPN *Gateway*, serta memungkinkan integrasi dengan sistem pendukung lainnya.



Gambar – Usulan Topologi

Komponen	Deskripsi
Web Proxy	Berperan sebagai pengontrol akses pengguna ke aplikasi, serta penghubung yang aman antara pengguna dan server
Firewall	Pada usulan topologi diatas terdapat 2 (dua) jenis *firewall*, diantaranya: 1. *Internal Firewall* (pengguna): melindungi jaringan internal dengan membatasi akses keluar yang tidak dikenal atau tidak memiliki izin. 2. *Internal Firewall* (data): melindungi akses dari DMZ (*Demilitarized Zone*) ke infrastruktur Aplikasi DIMS
Internet/WAN	Digunakan untuk menghubungkan pengguna ke server Aplikasi DIMS
VPN Gateway	Menyediakan akses aman bagi pengguna *remote* (misalnya dari luar kantor PLN) untuk terhubung ke sistem melalui koneksi terenkripsi dan terautentikasi.
Server Farm	Merupakan inti dari sistem DIMS yang terdiri dari 2 (dua) server berikut: • *Application Server*: Menjalankan *logic* dan fungsionalitas utama aplikasi. • Database Server: Menyimpan data yang dibutuhkan dan dihasilkan oleh aplikasi.
Surrounding Apps	Sistem pendukung eksternal yang terintegrasi dengan aplikasi utama, terdiri dari:

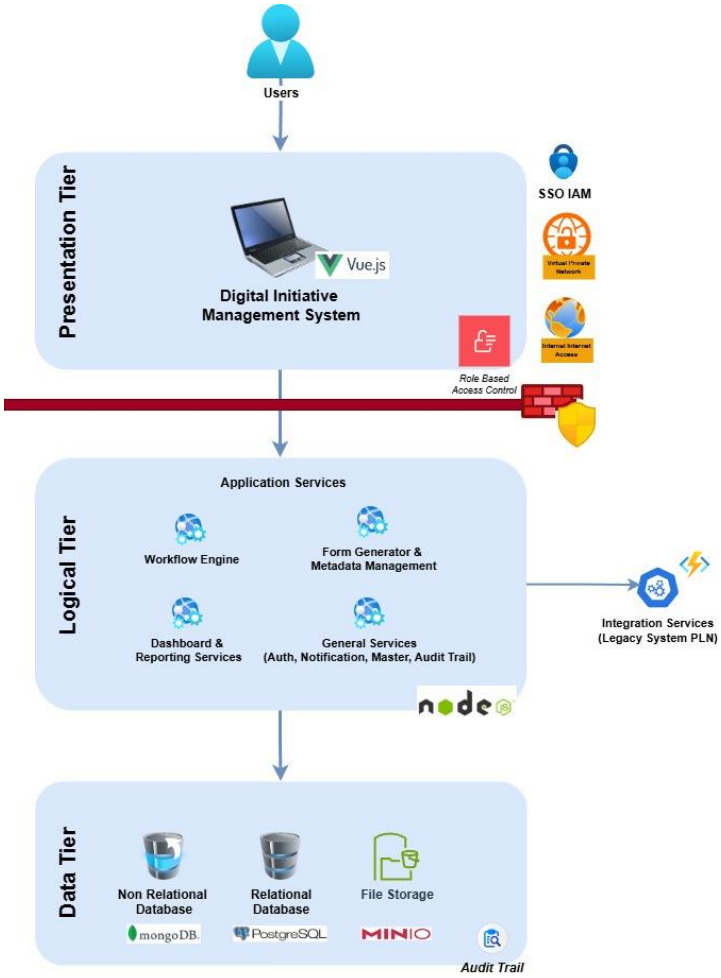
Komponen	Deskripsi
	• SSO Server: Mengelola autentikasi pengguna secara terpusat, sehingga tidak perlu login berulang kali untuk mengakses berbagai aplikasi yang terintegrasi. • Aplikasi Manajemen Surat (AMS): Proses penerbitan nomor Nota Dinas yang digunakan sebagai dasar pengajuan permohonan CR/JR.

3.3 Perancangan Arsitektur

Arsitektur sistem adalah desain atau struktur fundamental dari sebuah *software* aplikasi yang mencakup bagaimana berbagai komponen dan modul aplikasi saling berinteraksi, serta bagaimana aplikasi tersebut dikelola, dipelihara, dan dikembangkan seiring waktu. Dalam konteks pengembangan Aplikasi DIMS, arsitektur sistem dirancang agar mendukung skalabilitas, fleksibilitas, keamanan, serta kemudahan penggunaan oleh pengguna, khususnya dalam hal pengelolaan formula risiko dan analisis data historis. Arsitektur sistem ini menggambarkan alur pengolahan data dari sumber hingga penyajian dalam bentuk dashboard untuk pengguna.

Arsitektur ini terdiri dari beberapa tier dan komponen utama sebagai berikut:

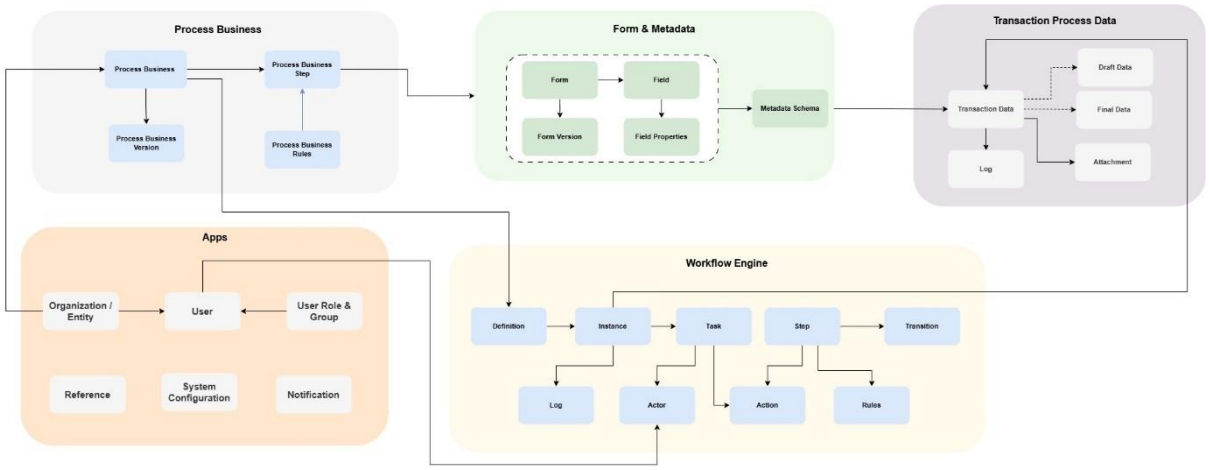
Tier	**Deskripsi**
Presentation	Menyediakan *interface* untuk interaksi *user* dengan sistem secara langsung.
Logical	Menangani logika bisnis, proses aplikasi, dan integrasi antar komponen sistem serta layanan eksternal
Data	Menyimpan, mengelola, dan menyediakan data yang dibutuhkan oleh system.



Gambar - Arsitektur Sistem

3.4 Perancangan Data Model

Tahap ini merupakan fase krusial yang bertujuan untuk menerjemahkan kebutuhan bisnis, dan strategi organisasi ke dalam struktur data yang sistematis, logis, dan mudah dikelola. Perancangan model data menjadi dasar utama dalam mendukung pelacakan performa strategis, meningkatkan pemantauan strategi, serta pengambilan keputusan berbasis data dalam DIMS.



Gambar – *Conceptual* Data Model

Beberapa aktivitas utama dalam tahap ini meliputi:

1. **Perancangan Skema Data (*Data Schema Design*):** Menentukan struktur tabel, entitas, dan relasi antar entitas berdasarkan pendekatan *star schema* atau *snowflake schema*. Skema ini mencakup:
   - *Conceptual Data Model* (CDM): Menggambarkan data yang akan dikelola oleh sistem dari sudut pandang bisnis. Model ini menunjukkan entitas utama yang terlibat, serta hubungan antar entitas tersebut.
   - *Physical Data Model* (PDM): Menerjemahkan CDM ke dalam struktur tabel, kolom, dan tipe data yang akan diimplementasikan dalam *database*.
2. **Pemetaan Data:** Proses menghubungkan elemen data dari berbagai sumber ke struktur data yang dirancang dalam model dengan tujuan menjamin kesesuaian data saat dilakukan integrasi, migrasi, atau pengolahan lebih lanjut.
   - Mengidentifikasi sumber data (misalnya: *database* lama, file eksternal, API)
   - Pemetaan data internal (misal: *workflow*, proses bisnis, form) ke struktur model.
   - Penyelarasan format, tipe data, dan standar penamaan.
3. **Penerapan Aturan Bisnis (*Business Rules Implementation*):** Penerapan aturan bisnis PLN ke dalam model data dan rancangan sistem untuk memastikan integritas data, konsistensi proses, serta kesesuaian dengan kebijakan dan prosedur operasional yang berlaku. Implementasi

dilakukan melalui validasi data, pengaturan relasi antar entitas, serta pembatasan dan kontrol pada setiap proses bisnis yang dikelola oleh sistem, dengan contoh sebagai berikut:

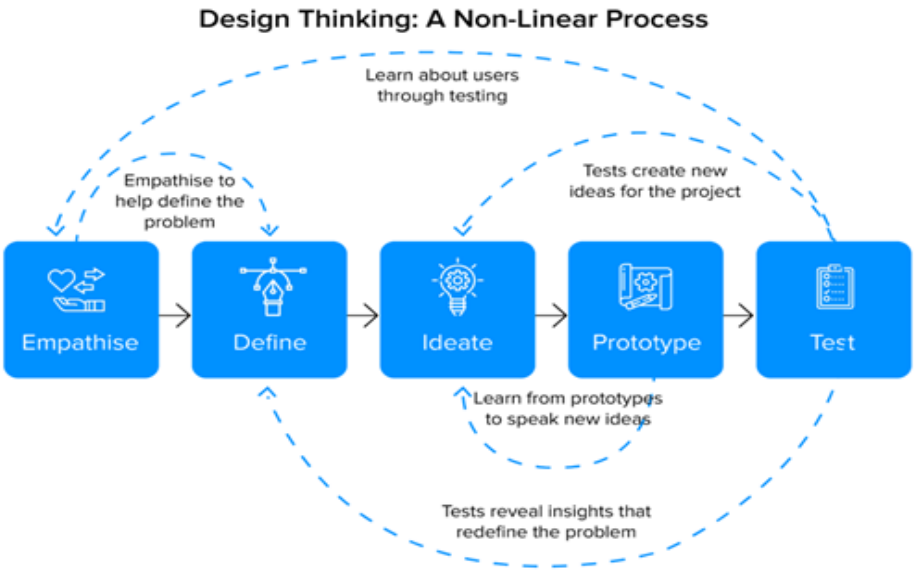
- Pengajuan Ide, *Change Request* atau *New Request* untuk aplikasi
- Formula perhitungan pada KPI.
- Hierarki organisasi untuk pembatasan akses berbasis *role*.

4. **Validasi & Dokumentasi**
   - Memastikan kelengkapan dan keakuratan relasi antar tabel.
   - Menyusun dokumentasi struktur data sebagai bagian dari acuan pengembangan lebih lanjut.

Dengan pendekatan ini, model data DIMS tidak hanya mendukung fungsi operasional aplikasi seperti manajemen user, tetapi juga memungkinkan konfigurasi modular untuk proses bisnis lainnya.

3.5 Perancangan Antar Muka

Dalam melakukan *improvement* pada desain UI/UX, kami menggunakan pendekatan dan metode yang berdasar pada prinsip *Design Thinking*, seperti pada gambar berikut:



Gambar 11 Proses *Design Thinking*

Berikut merupakan detail masing-masing proses desain yang kami ajukan:

1. *Empathize*
   a. Memahami alur proses bisnis eksisting.
   b. Memahami alur *user journey* dengan mengidentifikasi langkah-langkah yang dilakukan oleh *user* untuk memahami bagaimana pengguna berinteraksi dengan antarmuka dan proses yang ada.
   c. Memahami *pain point* atau *problem area* pada aplikasi eksisting dari sudut pandang UI/UX.

d. Mengidentifikasi dan memahami *branding guide design system/guidelines* yang digunakan oleh PLN.

2. *Define*
   a. Menentukan prioritas terhadap *pain point* atau *problem area* yang akan diselesaikan terlebih dahulu.
   b. Menentukan persona *user* atau kelompok *user* serta kebutuhan spesifik dari masing-masing kelompok *user* yang akan diberikan dari UI/UX aplikasi.
   c. Menentukan apakah diperlukan adanya perubahan mendasar dalam *user journey* dari aplikasi eksisting menuju aplikasi baru yang dikembangkan.
   d. Menentukan prinsip-prinsip dasar yang akan digunakan dalam pembuatan desain UI/UX *report* PLN.
3. *Ideate*
   a. Membuat alternatif-alternatif untuk adaptasi dan penyesuaian *branding guide* dari organisasi PLN, menjadi *branding guide* dan desain sistem aplikasi.
   b. Membuat alternatif-alternatif untuk menyelesaikan *pain point* atau *problem area*
4. *Prototype*
   a. Melakukan proses *wireframing* dan *low-fidelity mockup* untuk mendapatkan *feedback* dan validasi terhadap struktur informasi dan bentuk dasar dari UI aplikasi dan dashboard yang akan dibuat.
   b. Melakukan proses pembuatan *high-fidelity mockup* untuk mendapatkan *feedback* dan validasi terhadap gambaran produk akhir dari UI aplikasi.
   c. Merangkai *mockup* menjadi *prototype* interaktif yang bisa diakses oleh *user* secara *online*, sehingga meminimalisir kendala ruang dan waktu.
5. *Test*
   a. Menyelenggarakan sesi pengujian dengan user atau tim *counterpart* untuk mendapatkan *feedback* dari *prototype* interaktif.
   b. Melakukan perbaikan berdasarkan hasil diskusi dan *feedback* dari *user*.

3.6 Digital Workflow & Process Automation

3.6.1 *Workflow Engine*

Untuk pengembangan proses pengelolaan permohonan CR dan JR pada Aplikasi DIMS, Akhdani mengusulkan penerapan *Workflow Engine* berbasis *State Machine* dan *Metadata-Driven Workflow* untuk mengelola proses bisnis secara terstandarisasi, terukur, dan terdokumentasi. Pendekatan ini memungkinkan seluruh proses permohonan CR dan JR dijalankan secara konsisten mulai dari tahap pengajuan, review, persetujuan, implementasi, hingga penyelesaian.

Workflow Engine yang diusulkan tidak hanya berfungsi sebagai pengendali alur proses, namun juga sebagai mekanisme orkestrasi yang mengelola status proses, penugasan, SLA, notifikasi, audit trail, serta pengumpulan data operasional yang dapat digunakan untuk kebutuhan monitoring dan analisis *bottleneck* proses.

3.6.1.1 Prinsip Desain *Workflow Engine*

Workflow Engine dibangun berdasarkan beberapa prinsip utama sebagai berikut:

1. ***Metadata-Driven Workflow***

   Definisi proses bisnis, tahapan, *form*, aturan transisi, SLA, dan *assignment* disimpan dalam bentuk *metadata* sehingga perubahan proses dapat dilakukan melalui konfigurasi tanpa memerlukan perubahan kode program.

2. ***Versioned Configuration***

   Setiap proses bisnis, *workflow*, dan *form* memiliki mekanisme *versioning* yang terdiri atas status *Draft, Published*, dan *Archived*. Setiap *instance* proses akan menggunakan versi *workflow* yang aktif pada saat proses tersebut dibuat sehingga menjamin konsistensi proses dan histori perubahan.

3. ***State Machine Based Execution***

   Workflow Engine menjalankan proses berdasarkan konsep *state machine*, dimana setiap proses berpindah dari satu *state* ke *state* lainnya melalui *transition* yang telah ditentukan. Pendekatan ini memberikan kontrol yang lebih baik terhadap validasi proses, pengelolaan status, dan *auditabilitas* sistem.

4. ***Append-Only Audit Trail***

   Seluruh aktivitas *workflow* dicatat sebagai event yang bersifat *immutable* (tidak dapat diubah). Perubahan status tidak akan menimpa data sebelumnya sehingga seluruh histori proses dapat ditelusuri secara lengkap untuk kebutuhan audit maupun analisis kinerja.

5. **SLA *Management & Escalation***

   Pengaturan SLA per tahapan, monitoring *real-time*, dan eskalasi otomatis.

6. ***Assignment & Approval***

   Penugasan persetujuan berbasis *role/entitas* dan mekanisme persetujuan yang fleksibel.

7. ***Notification Engine***

   Notifikasi otomatis untuk aktor yang terlibat disetiap perubahan status tahapan *workflow*.



Gambar – Contoh Alur Proses

Nilai tambah:

- Fleksibel terhadap perubahan proses bisnis
- Monitoring SLA lebih teratur
- *Audit trail* lengkap dan transparan
- Identifikasi *bottleneck* lebih mudah
- Mendukung digitalisasi *end-to-end*

3.6.1.2 Komponen Utama *Workflow Engine*

Komponen utama yang diusulkan pada *Workflow Engine* meliputi:

Komponen	Deskripsi
Workflow Definition	Template proses bisnis yang telah dipublikasikan dan digunakan sebagai dasar pembentukan *instance* proses.
State	Tahapan proses yang merepresentasikan posisi terkini suatu permohonan.
Transition	Aturan perpindahan antar *state* yang menentukan alur proses yang diperbolehkan.
Guard	Validasi yang harus dipenuhi sebelum suatu transisi dapat dijalankan.
Action	Aktivitas otomatis yang dieksekusi setelah transisi berhasil dilakukan.
Task	Unit pekerjaan yang diberikan kepada pengguna atau kelompok pengguna tertentu.
Event Audit	Catatan aktivitas sistem yang digunakan untuk audit trail dan analisis proses.

3.6.1.3 Mekanisme Transition

Workflow Engine mendukung berbagai jenis transisi proses yang umum digunakan pada proses bisnis pengelolaan CR dan JR, antara lain:

Jenis Transisi	Fungsi
Submit	Mengirim permohonan ke tahapan berikutnya.

Jenis Transisi	Fungsi
Approve	Menyetujui proses dan melanjutkan ke state berikutnya.
Return Previous	Mengembalikan proses ke tahapan sebelumnya untuk perbaikan.
Return to Initiator	Mengembalikan proses kepada pengusul awal.
Return to Specific State	Mengembalikan proses ke tahapan tertentu sesuai aturan yang berlaku.
Reject	Menolak proses.
Reassign	Memindahkan penugasan kepada pengguna lain tanpa mengubah *state* proses.

Setiap transisi akan menghasilkan *event* audit yang mencatat informasi seperti pelaku proses, *state* asal, *state* tujuan, waktu pelaksanaan, komentar, serta alasan perubahan sehingga seluruh aktivitas dapat ditelusuri secara lengkap.

3.6.1.4 Status Workflow Instance

Selain mendukung berbagai jenis transisi proses, *Workflow Engine* juga menyediakan status (*instance status*) untuk menggambarkan kondisi keseluruhan proses yang sedang berjalan. Status ini digunakan untuk memudahkan pemantauan progres dan kondisi proses pada setiap tahapan. Adapun status yang dapat digunakan adalah sebagai berikut:

Status	Deskripsi
Draft	*Instance* dibuat tapi belum disubmit/*start*
In Progress	*Instance* sedang berjalan dan memiliki *active task*
Completed	*Instance* selesai normal
Rejected	*Instance* selesai karena *final rejection*
Cancelled	*Instance* dibatalkan

3.6.2 *Business Process Repository* (Probis *Collection*)

Akhdani mengusulkan pemanfaatan *Business Process Repository (Probis Collection)* sebagai sumber referensi proses bisnis yang terintegrasi dengan *Workflow Engine* pada DIMS. Melalui pendekatan ini, *workflow* yang dibangun tidak disusun secara terpisah, melainkan mengacu pada proses bisnis, aturan bisnis, serta relasi *entitas* yang telah tersedia pada *repository* proses bisnis perusahaan.

Implementasi ini memungkinkan setiap *workflow* memiliki keterkaitan yang jelas dengan proses bisnis yang berlaku, sehingga mendukung standarisasi proses, konsistensi *implementasi*, serta *traceability* antara proses bisnis dan pelaksanaan *workflow* pada DIMS.

3.6.2.1 Komponen Utama

Probis Collection yang dimanfaatkan oleh DIMS terdiri dari beberapa komponen utama sebagai berikut:

Komponen	Deskripsi
Organization / Entity	Struktur organisasi, unit kerja, stream, atau fungsi bisnis yang terlibat dalam suatu proses bisnis.
Process Business Repository (Probis)	Kumpulan proses bisnis yang menjadi referensi utama dalam pembentukan *workflow*.
Process Step	Tahapan proses yang menjadi dasar pembentukan state pada *Workflow Engine*.
Business Rule Management	Aturan bisnis yang digunakan untuk mendukung *assignment*, *approval*, dan transisi *workflow*.
Entity Mapping	Relasi antara *entitas* organisasi dengan proses bisnis dan tahapan yang terkait.
Process Version	Informasi versi proses bisnis yang digunakan sebagai referensi *workflow*.

3.6.2.2 *Lifecycle* Pengelolaan Probis

Setiap proses bisnis yang tersimpan pada *repository* memiliki siklus hidup pengelolaan sebagai berikut:

1. **Draft**

   Proses bisnis disusun atau diperbarui oleh *Business Analyst* atau *System Analyst* beserta definisi tahapan dan aturan bisnis yang terkait.

2. ***Review***

   Proses bisnis ditinjau oleh pemilik proses (*process owner*) dan *stakeholder* terkait untuk memastikan kesesuaian dengan kebutuhan bisnis.

3. ***Published***

   Proses bisnis telah disetujui dan dapat digunakan sebagai referensi dalam pembentukan *workflow* pada aplikasi.

4. ***Archived***

   Proses bisnis tidak lagi digunakan untuk kebutuhan baru, namun tetap dipertahankan sebagai referensi historis dan kebutuhan audit.



Gambar – Lifecycle Pengelolaan Probis

Pada implementasi DIMS, *Probis Collection* berfungsi sebagai *repository* proses bisnis yang bersifat ***read-only***, sehingga pengguna hanya dapat menggunakan proses bisnis yang telah berstatus *published* tanpa dapat melakukan perubahan terhadap definisi proses bisnis tersebut. Seluruh aktivitas operasional dan manipulasi data, seperti pengajuan, persetujuan, monitoring, serta pengelolaan dokumen, dilakukan pada *data layer* DIMS, sehingga integritas master proses bisnis tetap terjaga.

Melalui pendekatan ini, DIMS dapat memastikan bahwa setiap proses CR dan JR pengembangan aplikasi yang dijalankan telah selaras dengan proses bisnis yang berlaku, terdokumentasi dengan baik, dan dapat ditelusuri secara menyeluruh.

3.7 Form Generator & Metada Management

Untuk mendukung pengelolaan form yang fleksibel dan mudah dikembangkan, akan diterapkan ***Form Generator & Metadata Management*** berbasis metadata (*metadata-driven form*). Melalui pendekatan ini, struktur form, validasi, dan keterkaitannya dengan *workflow* dikelola melalui konfigurasi sehingga perubahan kebutuhan bisnis dapat diakomodasi tanpa memerlukan perubahan kode program secara signifikan.

Setiap form dapat digunakan kembali pada berbagai proses bisnis serta terintegrasi *dengan Workflow Engine* untuk mendukung pengelolaan data secara lebih terstandarisasi dan adaptif.

Untuk mendukung fleksibilitas pengelolaan form, solusi yang diusulkan mencakup pengelolaan *template form*, *metadata field*, pembentukan antarmuka secara dinamis, validasi data, integrasi

dengan *workflow*, serta pengelolaan data hasil pengajuan. Adapun kapabilitas yang akan dikembangkan meliputi:

1. ***Form Design, Configuration & Validation***

   Sistem menyediakan pengelolaan form berbasis *metadata* yang disimpan dalam format terstruktur dan memiliki mekanisme *versioning*. Setiap perubahan konfigurasi form akan menghasilkan versi baru sehingga konsistensi data dan histori penggunaan form tetap terjaga. Sistem mendukung berbagai tipe field seperti *text, textarea, number, date, select, radio button, checkbox, file upload*, serta dilengkapi validasi seperti *required, string, min/max value, regex, enum*, tipe file, dan ukuran file.

2. ***Reusable Form***

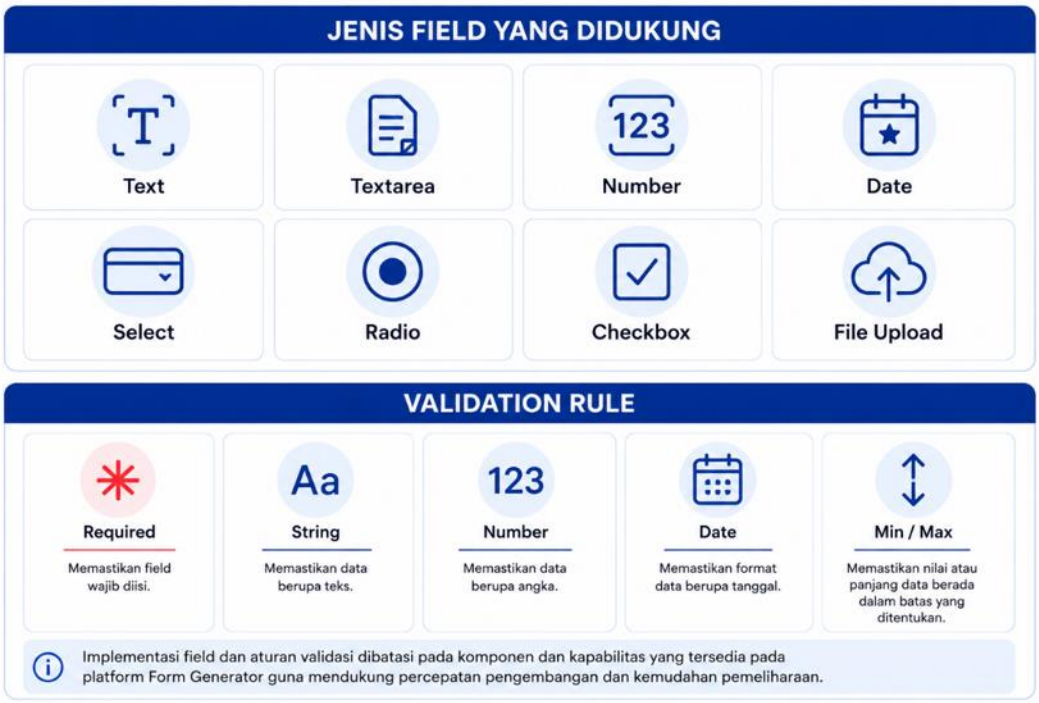
   Sistem mendukung penggunaan ulang (*reusable*) form pada berbagai proses bisnis dan tahapan *workflow*. Melalui pendekatan *metadata-driven*, form yang telah dibuat dapat digunakan kembali tanpa perlu dilakukan pengembangan ulang sehingga meningkatkan efisiensi pengelolaan dan konsistensi data antar proses.

3. ***Dynamic UI***

   Tampilan form dibangun secara dinamis berdasarkan *metadata* yang tersimpan pada sistem. Antarmuka dirancang menggunakan pendekatan yang sederhana dan mudah digunakan dengan dukungan pengelompokan *field* (*section*), label yang jelas, bantuan pengisian (*help text*), pesan validasi (*error message*), serta indikator progres pengisian form. Implementasi antarmuka difokuskan pada kesesuaian fungsionalitas dan konsistensi *style guide* yang telah disepakati bukan pada akurasi ***pixel-perfect***.

4. ***Submission Handling***

   Sistem menyediakan mekanisme penyimpanan data hasil pengisian form yang terintegrasi dengan *Workflow Engine*. Setiap data yang dikirim akan disimpan bersama informasi versi *template form* dan *snapshot metadata* yang digunakan pada saat pengisian, sehingga data historis tetap dapat ditampilkan dan ditelusuri meskipun terjadi perubahan struktur form pada versi berikutnya.



Gambar – Rule Form Generator & Metadata Management

3.8 Dashboard Monitoring & Report

Untuk mendukung kebutuhan monitoring, pengendalian proses, dan pelaporan kinerja secara *real-time,* akan diterapkan *Dashboard Monitoring & Reporting* yang terintegrasi dengan *Workflow Engine*. Dashboard ini dirancang berdasarkan *use case* pengelolaan *Change Request* (CR) dan *Job Request* (JR), sehingga setiap pengguna memperoleh informasi yang relevan sesuai peran dan tanggung jawabnya.

Selain mendukung monitoring operasional, dashboard juga menyediakan informasi terkait progres permohonan, pencapaian SLA, realisasi penyelesaian, serta laporan kinerja yang diperlukan untuk mendukung pengambilan keputusan berbasis data.

Dashboard yang akan dikembangkan meliputi:

1. ***Stream* BPO**

   Mendukung proses pengajuan dan monitoring permohonan CR/JR yang diajukan oleh BPO, yang mencakup:

   a. Pengajuan CR dan JR.

   b. Dashboard *Overview*.

2. ***Stream* MDG**

   Mendukung fungsi monitoring, evaluasi, dan pengendalian proses CR/JR pada level manajemen digital, yang mencakup:

   a. *Review* Pengajuan CR dan JR.

b. Dashboard *Overview*.

c. Dashboard Realisasi.

d. Dashboard Laporan KPI.

3. *Stream* PKK

Mendukung kebutuhan monitoring kinerja dan pelaporan KPI yang mencakup:

a. Dashboard Realisasi.

b. Dashboard Laporan KPI.

4. *Stream* STI

Menyediakan Dashboard Monitoring Permohonan untuk mendukung monitoring dan pengendalian permohonan CR dan JR yang sedang diproses oleh DIV STI, termasuk status permohonan, progres penyelesaian, dan pencapaian SLA.

3.9 File *Manager*

Fitur *File Manager* menyediakan repositori dokumen terpusat yang mengelola seluruh dokumen dari berbagai proses bisnis dan tahapan *workflow*, dilengkapi dengan *audit trail* untuk mencatat setiap aktivitas dokumen secara lengkap.

Sistem mendukung pencarian dokumen secara cepat berdasarkan nama, metadata, kata kunci, proses bisnis, maupun atribut lainnya, serta menerapkan pengendalian akses yang terstruktur untuk aktivitas *read*, *upload*, *replace*, dan *download* dokumen sesuai kewenangan.

3.10 Logging & Audit Trail

Logging sangat penting untuk memastikan bahwa data yang digunakan dalam pengambilan keputusan risiko selalu dalam kondisi valid dan *up-to-date*.

Dengan adanya sistem *logging* yang baik:

1. Kesalahan dapat dideteksi secara *real-time*, meminimalkan *downtime.*
2. Tim data dapat menelusuri akar masalah dengan cepat.
3. Mencegah penggunaan data yang *corrupt* atau tidak lengkap oleh aplikasi DIMS

Sedangkan, *audit trail* berfungsi mencatat semua perubahan data, termasuk siapa yang mengakses atau mengubah data dan kapan perubahan dilakukan. *Audit trail* ini sangat penting untuk kepatuhan terhadap kebijakan data *governance* dan keamanan, juga mendukung proses audit internal/eksternal yang membutuhkan jejak data historis.

3.11 Security, Governance, dan Audiatbility

3.11.1 *Implementasi Single Sign-On (SSO)*

Dalam pengembangan Aplikasi DIMS*,* penerapan *Single Sign-On* (SSO) menjadi komponen krusial untuk mendukung efisiensi akses dan pengelolaan keamanan pengguna. Berdasarkan dokumen Kerangka Acuan Kerja (KAK) dan *Business Requirement Document* (BRD), terdapat kebutuhan untuk mengintegrasikan sistem autentikasi yang terpusat, di mana pengguna dapat mengakses berbagai aplikasi di lingkungan PLN hanya dengan satu kali *login* melalui *Identity and Access Management (IAM)*. Setelah berhasil masuk, pengguna secara otomatis memiliki akses ke aplikasi-aplikasi tersebut tanpa perlu melakukan autentikasi ulang. Sistem SSO ini dirancang menggunakan *protokol* OAuth 2.0 dan mendukung integrasi dengan identitas digital yang dikelola oleh PLN.

3.11.2 Pengelolaan Hak Akses Pengguna

3.11.2.1 PENGATURAN AKSES PENGGUNA BERDASARKAN PERAN (RBAC)

Penerapan ***Role-Based Access Control*** **(RBAC)** dilakukan untuk mengatur hak akses pengguna berdasarkan peran (*role*) yang telah ditetapkan dalam sistem. Melalui mekanisme ini, setiap pengguna tidak diberikan akses secara individual, melainkan melalui peran yang memiliki hak dan tanggung jawab tertentu. Pendekatan ini memudahkan proses administrasi, menjamin konsistensi, serta meminimalkan risiko kesalahan pemberian akses.

Adapun langkah penerapan RBAC yang akan kami lakukan yaitu:

1. Identifikasi peran (*role*) sesuai fungsi bisnis seperti *Administrator, Approver*, Operator, dan *Viewer*.
2. Pemetaan fitur dan aksi pada setiap modul, misalnya *create, read, update, delete, approve,* dan *export*.
3. Definisikan daftar *permission* berdasarkan aksi yang telah diidentifikasi sebagai dasar kontrol akses.
4. Buat *matriks Role–Permission* untuk menentukan hak akses masing-masing peran.
5. *Assign role* ke *user* sesuai tanggung jawab dan unit kerjanya.
6. Implementasikan kontrol akses menggunakan *middleware* atau *policy* di sisi aplikasi.
7. Lakukan pengujian hak akses untuk memastikan pengguna hanya dapat melakukan aksi sesuai perannya.

Berikut pembagian *user role* untuk pengembangan aplikasi DIMS:

1. ***Stream* BPO**

Role	***Module***	
	Dashboard Overview	**Pengajuan CR dan JR**
BPO	R,W	R,W
DIV MDG	-	-
DIV STI	-	-
DIV PKK	-	-
Administrator Sistem	R,W	R,W

2. ***Stream* DIV MDG**

Role	***Module***			
	Dashboard Overview	**Review Pengajuan CR dan JR**	**Dashboard Realisasi**	**Dashboard Laporan KPI**
BPO	-	-	-	-
DIV MDG ITBP	R,W	R,W	R,W	R,W
DIV MDG non ITBP	R	-	R	R,W
DIV STI	-	-	-	-
DIV PKK	-	-	R	R
Administrator Sistem	R,W	R,W	R,W	R,W

3. ***Stream* DIV STI**

Role	***Module***
	Dashboard Monitoring Permohonan CR dan JR
BPO	-
DIV MDG	-
DIV STI	R,W
DIV PKK	-
Administrator Sistem	R,W

4. *Stream* DIV PKK

Role	*Module*	
	Dashboard Realisasi	**Dashboard Laporan KPI**
BPO	-	-
DIV MDG	-	-
DIV STI	-	-
DIV PKK	-	-
Administrator Sistem	R,W	R,W

3.11.2.2 PENERAPAN PRINSIP KEAMANAN *LEAST PRIVILEGE*

Selain RBAC, sistem juga mengadopsi **Prinsip *Least Privilege***, yaitu konsep keamanan yang memastikan setiap pengguna, proses, atau layanan hanya memiliki hak minimum yang diperlukan untuk menjalankan fungsinya. Prinsip ini diterapkan di seluruh lapisan sistem, mulai dari akses pengguna, layanan aplikasi, hingga konfigurasi *server* dan *database*. Penerapan prinsip ini bertujuan untuk membatasi ruang lingkup potensi pelanggaran, mengurangi risiko kesalahan operasional, serta meningkatkan stabilitas dan keamanan sistem secara keseluruhan. Dengan demikian, jika terjadi kompromi pada satu akun atau komponen, dampaknya dapat diminimalkan karena hak akses yang dimiliki bersifat terbatas. Berikut hak akses untuk setiap *role* dalam Aplikasi DIMS.

Penerapan prinsip ini dilakukan melalui beberapa langkah berikut:

1. Pembatasan ruang lingkup data (data *scope*), yaitu setiap pengguna hanya dapat mengakses data yang menjadi tanggung jawabnya. Sebagai contoh, pengguna dengan peran *Operator* hanya dapat melihat atau mengelola data milik unit atau wilayahnya sendiri, misalnya berdasarkan *unit_id* atau *region_id*.
2. Pembatasan tingkat aksi (*action level*), di mana akses terhadap fungsi tertentu dibatasi sesuai peran. Misalnya, pengguna dengan peran *Viewer* hanya dapat melihat ringkasan laporan tanpa memiliki izin untuk mengunduh dokumen yang bersifat sensitif.
3. Penerapan kebijakan tambahan (*policy-based restriction*), yang mengatur batas kewenangan berdasarkan hierarki organisasi. Sebagai contoh, *Approver* hanya dapat melakukan persetujuan terhadap data yang berada dalam lingkup unit kerja di bawah pengawasannya.

Dengan pendekatan tersebut, sistem tidak hanya mengatur akses berdasarkan peran, tetapi juga menegakkan prinsip keamanan berlapis yang adaptif terhadap struktur organisasi dan kebijakan tata kelola TI perusahaan.

3.12 Strategi Pengujian

Dalam proses pengembangan Aplikasi DIMS, strategi pengujian difokuskan pada aspek teknis dan fungsional sistem untuk memastikan bahwa solusi yang dibangun telah sesuai dengan kebutuhan bisnis, proses operasional, serta spesifikasi yang telah disepakati. Strategi ini dibagi menjadi tiga lapisan: ***manual testing*** sebagai verifikasi awal, ***automated code review*** menggunakan *SonarQube* untuk menjamin kualitas struktur kode, penerapan *best practice* dan *maintainability* kode, ***manual code review*** untuk memastikan kualitas implementasi dan kesesuaian desain aplikasi, serta ***automated testing*** guna memastikan kestabilan dan konsistensi sistem secara berkelanjutan.

3.12.1 *Manual Testing*

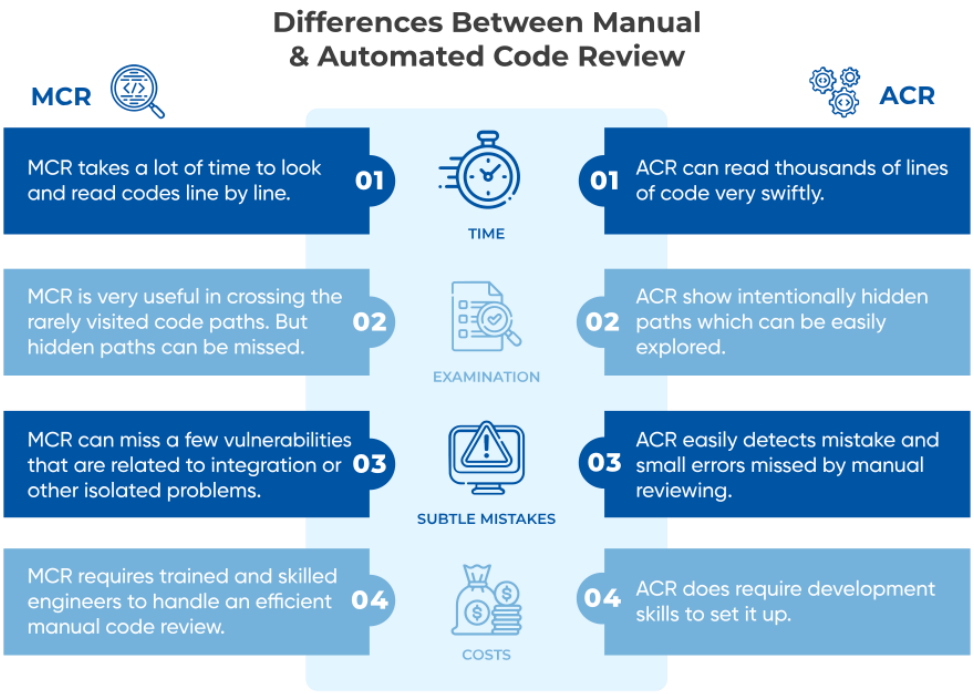
Manual testing dilakukan untuk memastikan bahwa setiap fitur dan alur porses dalam aplikasi DIMS berjalan sesuai dengan kebutuhan bisnis dan rancangan fungsional. Pengujian ini mencakup validasi terhadap logika pemodelan risiko dan antarmuka pengguna.

Fokus Pengujian *Manual Testing*:

- **Uji Fungsional**: Memastikan bahwa setiap fitur aplikasi bekerja sesuai dengan spesifikasi fungsional yang telah dirancangm seperti pengujian aturan validasi *input*, *field* wajib dan alur kerja fitur.
- **Uji Integrasi Modul:** Pengujian interkasi dan komunikasi antar modul dalam aplikasi DIMS.
- **Uji *User Acceptance* (UAT):** Melibatkan pengguna aplikasi dengan *role* yang berbeda (xxxx) untuk menguji aplikasi sesuai dengan kebutuhan operasional.

3.12.2 *Automated Code Review*

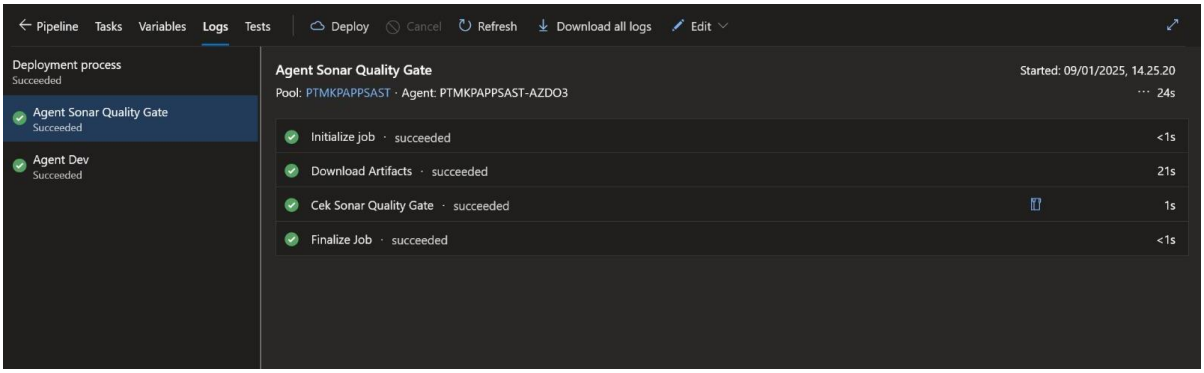
Automated code review merupakan salah satu proses yang harus dilakukan pada fase pengembangan aplikasi yang berfungsi untuk meningkatkan kualitas dari aplikasi yang dibuat. *Code review* biasa dilakukan oleh *team leader*/senior developer dari struktur tim pengembangan aplikasi.



Gambar – Perbedaan Manual & Automated Code Review

Berbeda dengan proses *code review* manual yang mengandalkan peninjauan langsung oleh tim pengembang, ***automated code review*** digunakan untuk menjaga kualitas struktur dan keamanan kode sejak awal. Setiap perubahan kode dapat diperiksa otomatis menggunakan *SonarQube*

SonarQube merupakan *platform open-source* untuk analisis statis kode yang mampu mendeteksi *bug*, *code smell*, kerentanan keamanan, dan duplikasi. Dukungan terhadap berbagai bahasa pemrograman dan integrasi dengan *version control* seperti *GitLab* menjadikannya ideal dalam proses *CI/CD*.



Gambar – Sonar Quality Gate

Fokus *Automated Code Review:*

- **Kepatuhan Standar Kode:** Konvensi penamaan, struktur folder, dan dokumentasi kode.
- **Pendeteksian *Bug* & *Code Smell:*** Menghindari duplikasi logika dan potensi masalah tersembunyi.

- **Keamanan Kode:** Mendeteksi potensi kerentanan terhadap injeksi, *XSS*, dan praktik *insecure coding*.
- **Integrasi dengan *CI/CD*:** Semua review dilakukan secara otomatis sebelum proses *merge*.

3.12.3 *Manual Code Review*

Dalam pelaksanaan pengembangan sistem ini, kami juga menerapkan mekanisme manual *code review* yang berjenjang sebagai bentuk komitmen terhadap kualitas dan tata kelola TI (*IT Governance*). Proses ini dipimpin langsung oleh *Senior Developer* untuk memastikan bahwa setiap perubahan kode yang akan diimplementasikan tidak hanya berfungsi secara fitur, tetapi juga mematuhi standar keamanan, performa, dan kepatuhan (*compliance*) yang berlaku di lingkungan PLN.

1. ***Quality & Risk Management***

   Seluruh hasil pengembangan akan melalui proses review oleh personel senior sebagai bagian dari mekanisme pengendalian mutu. Proses ini bertujuan untuk memastikan bahwa solusi yang dibangun memenuhi standar kualitas yang telah ditetapkan, baik dari aspek fungsional, performa, keamanan, maupun *maintainability*. Dengan adanya pemeriksaan berlapis, potensi risiko teknis dapat diidentifikasi dan ditangani lebih awal sehingga mengurangi kemungkinan terjadinya permasalahan pada saat implementasi maupun operasional sistem.

2. ***Auditability***

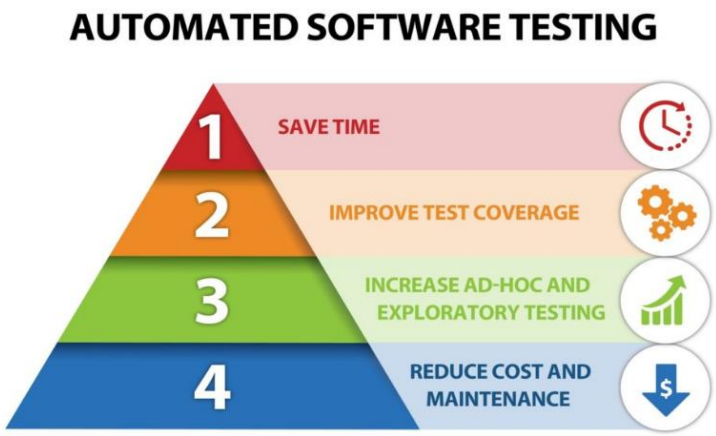
   Setiap perubahan yang dilakukan selama proses pengembangan terdokumentasi secara sistematis dan dapat ditelusuri dengan jelas. Dokumentasi tersebut mencakup riwayat perubahan, pihak yang melakukan pengembangan, serta proses review. Mekanisme ini mendukung penerapan tata kelola teknologi informasi yang baik, meningkatkan akuntabilitas pelaksanaan proyek, serta memudahkan kebutuhan audit, monitoring, dan evaluasi di masa mendatang.

3. ***Transfer of Knowledge* dan Standarisasi**

   Proses review tidak hanya berfungsi sebagai kontrol kualitas, tetapi juga menjadi sarana pembelajaran dan transfer pengetahuan di dalam tim. Melalui diskusi teknis dan pemberian masukan secara berkelanjutan, pemahaman terhadap standar pengembangan, arsitektur solusi, dan praktik terbaik dapat disebarkan secara merata kepada seluruh anggota tim. Pendekatan ini membantu meningkatkan kapabilitas tim, mengurangi ketergantungan pada individu tertentu, serta mendukung keberlanjutan pengelolaan dan pengembangan aplikasi dalam jangka panjang.

3.12.4 *Automated Testing*

Automated testing dilakukan untuk menjamin bahwa seluruh fungsi sistem tetap berjalan stabil selama *reengineering* berlangsung maupun setelah *deployment*.



Manfaat *Automated Testing*

Karena logika bisnis tidak berubah, *test case* yang dibuat berfokus pada **kesetaraan fungsionalitas dan kestabilan sistem** melalui tiga pendekatan utama: *unit testing, Frontend Perfomance Testing dan end-to-end testing*.

1. *Unit Testing*

Unit testing merupakan tahap paling dasar dalam *automated testing* yang berfokus pada **pengujian *logic* atau fungsi**. Tujuannya adalah untuk memastikan bahwa setiap *unit* (seperti fungsi atau metode) berjalan sesuai harapan dan dapat berdiri sendiri tanpa bergantung pada komponen eksternal lainnya. Pada Aplikasi DIMS, *unit testing* digunakan dalam komponen berikut:

a. ***Service Backend***

Untuk modul *backend* yang dibangun dengan Golang, **Vitest** digunakan untuk menguji *service-level logic*. Fokus utamanya adalah memastikan fungsi-fungsi seperti perhitungan risiko, validasi input, serta mekanisme *logging* dan *error handling* berjalan konsisten. Setiap fungsi diuji secara *isolated* untuk menghindari *dependency* antar modul selama pengujian.

b. Pengujian secara manual maupun otomatis, manual dilakukan oleh setiap *programmer* dan otomatis melalui *pipeline CI/CD* menggunakan *Gitlab Runner* untuk mendeteksi *error* sejak tahap awal. Dengan pengujian ini, setiap perubahan kode dapat dipastikan tidak mengganggu fungsionalitas inti di level metode.

2. *Frontend Performance Testing*

Frontend performance testing bertujuan untuk mengukur **kinerja antarmuka pengguna** (*UI*) dari sisi pengalaman pengguna secara teknis, terutama dalam hal **kecepatan muat halaman**, **interaktivitas**, dan **aksesibilitas**. Meskipun proses bisnis tidak mengalami perubahan, tetapi memerlukan evaluasi

performa untuk memastikan aplikasi tetap ringan, responsif, dan nyaman diakses di berbagai perangkat.

Pengujian dilakukan menggunakan ***Lighthouse Chrome***, yaitu alat analisis performa berbasis *browser* yang mampu memberikan penilaian terhadap beberapa aspek utama *UI*, antara lain:

- ***Performance***: waktu muat halaman (*load time*), *first contentful paint*, *time to interactive*
- ***Accessibility***: keterbacaan elemen, penggunaan warna, kontras, dan navigasi *keyboard*
- ***Best Practices***: pemanfaatan *HTTPS*, *caching*, serta pengelolaan *resource frontend*
- ***SEO* (opsional)**: untuk pengukuran dasar jika aplikasi dibuka melalui *web browser* publik

Laporan *Lighthouse* ditampilkan dalam bentuk skor dengan *breakdown* per aspek, yang membantu tim *frontend* melakukan penyempurnaan jika ditemukan potensi *bottleneck* atau inefisiensi pada *UI*. Pengujian ini penting dilakukan secara berkala, terutama setelah perubahan besar pada komponen tampilan atau integrasi baru.

3. *End-to-End Testing*

End-to-end testing merupakan tahapan tertinggi dalam *automated testing* karena mensimulasikan alur penggunaan nyata dari perspektif pengguna akhir. Tujuan utamanya adalah memastikan bahwa **setiap interaksi pengguna** dari awal hingga akhir berjalan mulus dan menghasilkan *output* yang diharapkan. Dalam aplikasi DIMS, *end-to-end testing* mencakup

3.13 Strategi *Deployment*

Dalam menjamin keberhasilan implementasi aplikasi, strategi *deployment* disusun secara terstruktur dengan pendekatan kolaboratif dan berorientasi pada keberlanjutan sistem. Strategi ini mencakup dua aspek utama: pengelolaan versi kode *(version control)* dan penerapan sistem ke lingkungan produksi *(deployment & operational).*

3.13.1 *Version Control*

Version control memungkinkan *developer* atau anggota tim lainnya untuk membandingkan *file*, mengidentifikasi perbedaan, dan menggabungkan perubahan yang diperlukan sebelum melakukan aksi apa pun. Pembuatan versi juga merupakan cara yang dapat digunakan untuk melacak pembuatan aplikasi dengan dapat mengidentifikasi versi mana yang saat ini dilakukan *development*, *quality assurance,* dan *production*.

Adapun beberapa manfaat dari dilakukannya *versioning* ini adalah:

1. *Traceability*

   Dengan dilakukannya *versioning* maka perbedaan atau kekurangan *file* dapat dengan mudah dilacak sehingga dapat meminimalisir kesalahan antar anggota tim ataupun *human error.*

2. *Identifiability*

   Versioning memudahkan untuk dilakukan identifikasi bagaimana atau hal apa yang berubah sampai dengan identifikasi waktu, *file* dan kontributor nya.

3. *Clarity*

   Beberapa dokumen nantinya akan lebih mudah dibedakan, sehingga dokumen atau *file* terbaru dapat dengan mudah dikenali.

4. *Reduced duplication*

   Dengan dilakukan *versioning* maka hal tersebut akan mengurangi *file* yang duplikat.

5. *Reduced errors*

   Sama halnya dengan poin diatas, dengan *versioning* juga nantinya akan jauh mengurangi kemungkinan *error* pada aplikasi.

Git adalah *version control system* yang terdistribusi yang dapat melacak perubahan *source-code* selama pengembangan *software*. *Git* dirancang untuk mengkoordinasikan dan melacak perubahan pada *file* aplikasi di antara anggota tim. *Git* bersifat bebas dan *open source*.

Fitur *Git*:

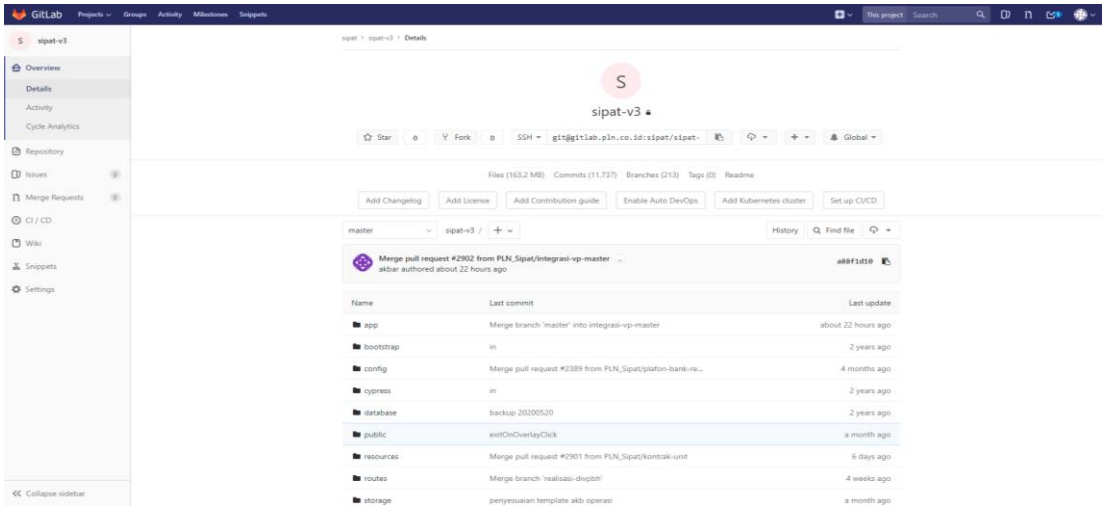
1. Terdistribusi

   Maksud dari terdistribusi adalah *source* tidak berada pada satu server yang terpusat, melainkan *source code* dikloning ke mesin lokal masing-masing *programmer*. Sehingga tidak mengganggu *source code* utama yang ada di *server* pusat meskipun pekerjaan para *programmer* belum selesai. Setelah selesai barulah *source code* di-*push* ke server *repository* pusat.

2. *Branching* dan *Merging*

   Branching adalah membuat cabang berupa *versi source* code yang bisa berbeda-beda dan independen satu sama lain. Sehingga tidak mengganggu modul lain dalam satu *repository*.

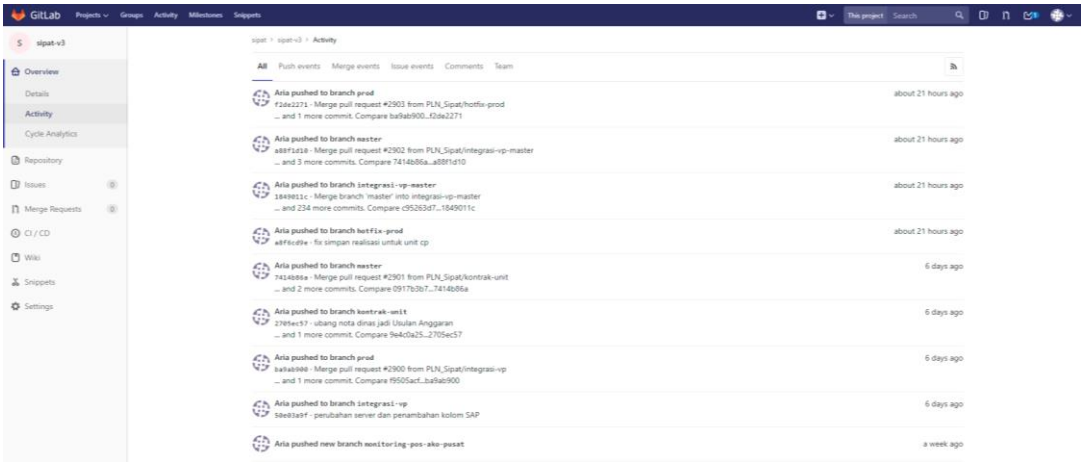
   Sedangkan *margin* adalah menggabungkan perubahan-perubahan cabang-cabang tersebut pada satu *branch*, biasanya dilakukan pada *branch master*, sehingga *branch master* adalah *branch* yang mempunyai posisi *source code* paling terbaru dan terlengkap.



Tampilan Git

Keunggulan dari *Git* :

1. *Design* yang sederhana
2. Mendukung dalam pengembangan *non-linear* (pengembangan paralel)
3. Sistem terdistribusi, *peer to peer*
4. Mendukung dalam proyek besar seperti *Kernel Linux*
5. Akses menggunakan *command line*
6. Penyimpanan murni berbasis *file*



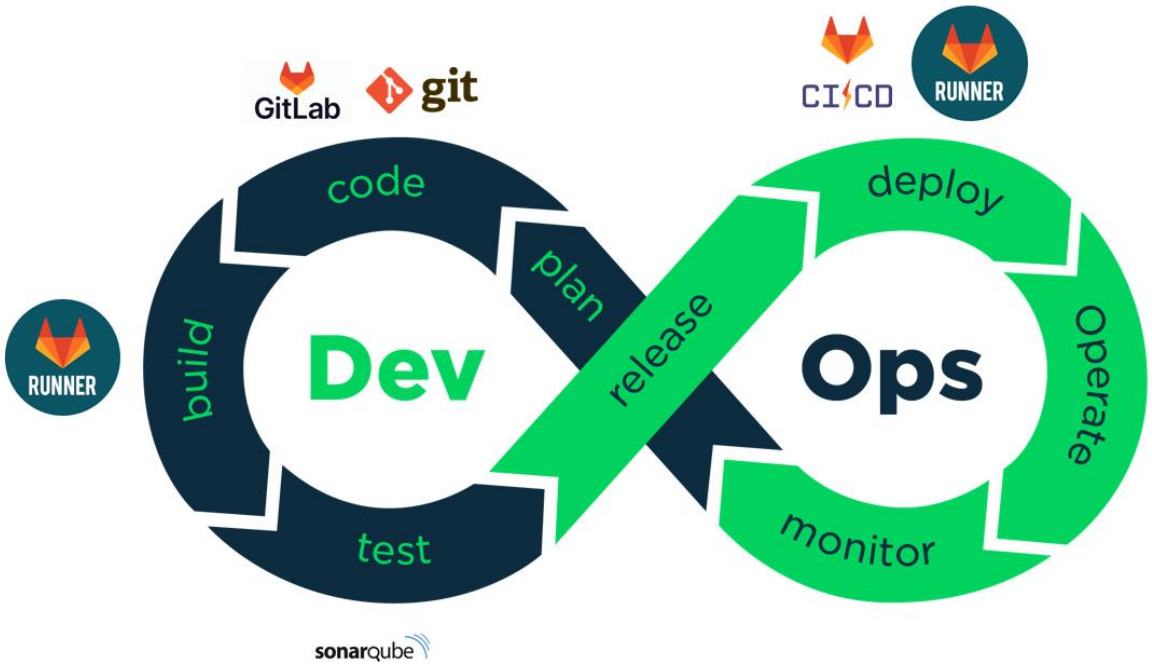
Tampilan Git

3.13.2 *Gitlab Runner*

Untuk memastikan proses *deployment* berjalan secara otomatis, cepat, dan rapi, maka digunakan *GitLab Runner. GitLab Runner* adalah komponen pendukung dari *GitLab CI/CD* yang bertugas untuk menjalankan perintah-perintah otomatis (*job*) yang sudah didefinisikan oleh tim *programmer*, seperti proses *build, test,* dan *deployment* aplikasi.

Setiap kali *programmer* meng*update* aplikasi, *GitLab Runner* akan langsung bekerja dengan mengambil versi terbaru dari aplikasi, menjalankan pengecekkan yang diperlukan, lalu melakukan *deployment* ke *server* secara otomatis.

3.13.3 Strategi *Deployment*



Gambar 12 Strategi *Deployment*

Adapun strategi *deployment* yang dilakukan berbasis *DevOps* dengan tahapan seperti berikut:

1. ***Plan***, dilakukan oleh tim bisnis dan tim pengembang untuk mendefinisikan *backlog* sesuai kebutuhan aplikasi.
2. ***Code***, hasil penulisan kode disimpan ke dalam repository Gitlab. Sedangkan Git digunakan sebagai sistem version control untuk mencatat setiap perubahan kode secara terstruktur.
3. ***Build***, dilakukan menggunakan *GitLab Runner* yang merupakan proses awal dari *pipeline CI/CD* yang dilakukan secara otomatis setelah ada perubahan kode.
4. ***Test***, pengujian yang dilakukan mencakup:
   a. *Vitest*: digunakan untuk *unit testing*.
   b. SonarQube: digunakan untuk *automated code review*

   Proses pengujian ini dijalankan otomatis melalui *GitLab Runner*.
5. ***Release****, build* yang berhasil lolos pengujian akan masuk ke tahap *deployment*.
6. ***Deploy***, proses *deployment* ke *production* yang dijalankan melalui *GitLab Runner*.

Keunggulan dari proses *DevOps*:

1. Proses *build, test,* analisis kualitas kode, dan *deployment* dilakukan secara otomatis dalam satu alur. Ini mengurangi kesalahan manual dan mempercepat waktu rilis.
2. Adanya pengujian dan *quality code* sehingga kode memenuhi standar kualitas sebelum dilakukan *deployment*.
3. *Deployment* ke *server staging* dilakukan dengan cepat dan aman, tanpa perlu proses manual atau konfigurasi ulang.

3.13.4 *Deployment and Operational*

Deployment Aplikasi DIMS tidak hanya berfokus memindahkan sistem ke server, melainkan merupakan fase kritis dalam menjamin keberhasilan implementasi sistem digital yang strategis. Proses deployment dilakukan secara bertahap dan terkendali.

Pada tahap staging, seluruh fitur baru yang dikembangkan dilakukan simulasi dalam lingkungan yang menyerupai kondisi operasional PLN. Tahapan ini memastikan bahwa hasil pengujian (UAT) sudah ditindaklanjuti dan diuji ulang.

Setelah sistem dinyatakan stabil dan sesuai kebutuhan pengguna, dilakukan proses *Put on Production* (POP) di server utama PLN. Proses ini mencakup:

- Instalasi akhir dan konfigurasi sistem sesuai standar infrastruktur PLN;
- Monitoring performa pasca-produksi, terutama untuk modul yang bersifat *real-time*;
- Mendukung dan melakukan pendampingan ke Tim Teknis IT PT PLN (Persero)

Strategi ini tidak hanya memastikan transisi yang mulus dan bebas gangguan, tetapi juga menjamin bahwa seluruh fitur strategis Aplikasi DIMS dapat berfungsi secara optimal dan siap mendukung efisiensi serta transparansi pengelolaan jaminan di lingkungan PLN.

4 TAHAPAN PENGEMBANGAN APLIKASI

Untuk rangkaian pelaksanaan pekerjaan mulai dari identifikasi kebutuhan hingga penyelesaian dan implementasi sistem, akan dijelaskan pada uraian berikut.

4.1 Analisis Kebutuhan & Perancangan *Design*

Tahapan Analisis Kebutuhan dan Perancangan Desain merupakan pondasi utama dalam pengembangan Aplikasi DIMS. Dalam dokumen BRD, tahap ini menjadi titik krusial dalam memastikan bahwa sistem yang dikembangkan benar-benar sesuai dengan kebutuhan PLN, baik dari sisi strategis, teknis, maupun operasional. Proses ini dilakukan secara kolaboratif antara DIV MDG sebagai *Business*

Process Owner (BPO) dan DIV STI sebagai *counterpart* teknis, bersama tim pengembang. Pada tahap ini, fitur-fitur dan operasi-operasi pada sistem akan dideskripsikan secara detail yang mencakup:

a. *Requirements Gathering* & *Gap Analysis*

   Pengumpulan kebutuhan dilakukan melalui diskusi dan analisis terhadap dokumen KAK dan BRD. Proses *gap analysis* dilakukan terhadap sistem eksisting.

b. Studi Eksisting Sistem

   Melakukan kajian terhadap aplikasi, proses bisnis, serta mekanisme pengelolaan permohonan yang saat ini berjalan. Kegiatan ini bertujuan untuk memahami alur proses, peran pengguna, kebutuhan data, integrasi sistem, serta kendala yang dihadapi pada kondisi eksisting

c. Perancangan Arsitektur

   Menyusun desain arsitektur aplikasi yang mencakup komponen *frontend*, *backend*, *database*, *workflow engine*, integrasi sistem, penyimpanan dokumen, serta mekanisme komunikasi antar layanan. Arsitektur dirancang untuk mendukung skalabilitas, keamanan, fleksibilitas, dan kemudahan pengembangan di masa mendatang.

d. Perancangan Data Model

   Menyusun model data yang mendukung pengelolaan proses bisnis, *workflow*, form dinamis, *metadata*, pengguna, *notifikasi*, *audit trail*, serta data referensi lainnya. Perancangan dilakukan untuk memastikan konsistensi, integritas, dan kemudahan pengelolaan data pada seluruh siklus proses permohonan.

e. Perancangan Antarmuka

   Menyusun rancangan antarmuka pengguna yang intuitif dan mudah digunakan, mencakup dashboard monitoring, form pengajuan, halaman persetujuan, pengelolaan *workflow*, serta fitur administrasi sistem. Perancangan dilakukan dengan mempertimbangkan kemudahan penggunaan dan pengalaman pengguna yang optimal.

f. Setup dan Konfigurasi *Environment*

   Menyiapkan dan mengonfigurasi lingkungan pengembangan (*development environment*) dan *deployment* yang meliputi *repository source code*, *database*, server aplikasi, *pipeline deployment*, serta *tools* pendukung pengembangan lainnya.

4.2 Aplikasi (*Development*)

Secara umum, tahapan pengembangan aplikasi mencakup *workflow engine*, *form generator & metadata management*, dashboard, file manager, *authenticacation*, *notification*, *history* perubahan data dan *audit trail* dan master proses bisnis *repository*, master tahapan, master *entitas*, serta managemen user. berdasarkan perkembangan level user untuk mendukung proses bisnis pengelolaan jaminan secara lebih terintegrasi dan efisien. Lingkup pengembangan meliputi:

Modul Aplikasi DIMS



Gambar - Pengembangan Aplikasi

Tujuan utama dari pengembangan Aplikasi DIMS adalah untuk memperkuat pondasi sistem dan meningkatkan kemampuan integrasi serta skalabilitas dalam mendukung pengelolaan jaminan secara digital. Proyek ini dirancang agar selaras dengan kebijakan transformasi digital PLN, dengan harapan dapat menjembatani kesenjangan antara proses manual sebelumnya dengan solusi yang lebih otomatis, aman, dan terintegrasi. Setiap fitur dikembangkan untuk memastikan fleksibilitas, kemudahan pemeliharaan, serta kemampuan adaptasi terhadap kebutuhan masa depan berdasarkan praktik terbaik dan wawasan berbasis data.

Pengembangan dan implementasi fitur pada Aplikasi DIMS dilakukan secara terstruktur dan berfokus pada kebutuhan bisnis yang telah teridentifikasi dalam dokumen BRD dan KAK. Proses ini dilaksanakan secara iteratif dengan pendekatan Agile. Fitur-fitur utama yang dikembangkan meliputi:

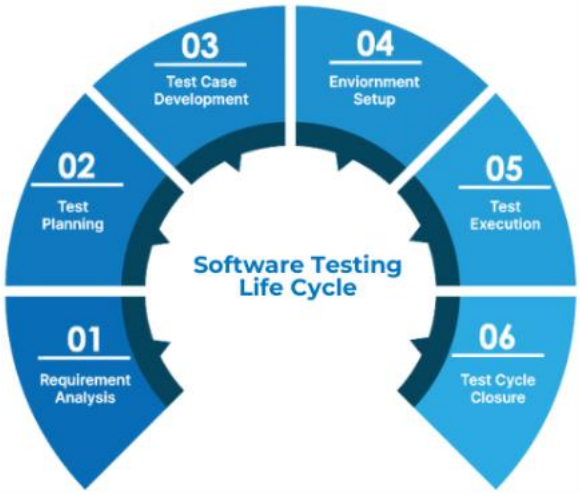
1. *Workflow Engine*
2. *Form Generator & Metadata Management*
3. *Authentication*
4. *File Manager*
5. Integrasi dengan *Legacy System* PLN

6. *History* Perubahan Data & *Audit Trail*
7. Dashboard
8. Master Proses Bisnis *Repository*
9. Master Tahapan
10. Master Entitas
11. Master *User Role* dan *User Group*
12. Master *User Management & Assign Role*
13. Master *System Configuration*
14. Master *Reference*

4.3 Review Hasil Pengembangan Aplikasi

Pada tahap ini dilakukan pertemuan *weekly meeting* secara *hybrid* dengan *stakeholder* untuk menunjukkan hasil pekerjaan dan meminta masukan pengguna. *Feedback* dari pengguna digunakan untuk memperbaiki atau menambah item di *backlog*.

Sebelum dilakukan sprint *review* dengan *stakeholder*, tim pengembang juga melakukan pengujian operasional penggunaan sistem yang sebenarnya (bukan simulasi), untuk menguji sistem secara keseluruhan. Pengujian yang dilakukan secara manual sesuai dengan kebutuhan dan iterasi yang berlangsung selama pengembangan. Adapun proses pengujian yang dilakukan sebagai berikut:



Gambar - Langkah Pengujian Aplikasi

Berikut ini beberapa pengujian yang dilakukan untuk memastikan bahwa sistem memenuhi kebutuhan dan siap digunakan dalam lingkungan sebenarnya.

1. **Pengujian & Perbaikan Internal**

Tahap awal pengujian yang dilakukan oleh tim internal pengembang sistem (*developer* dan QA internal) untuk memastikan fitur yang dikembangkan sesuai spesifikasi dan dapat berjalan dengan stabil. Aktivitas-aktivitas yang dilakukan adalah:

- Membuat rencana simulasi.
- Melakukan simulasi dan uji coba.
- Perbaikan hasil pengujian

2. Pengujian *Security* Aplikasi Internal

Pengujian *security* aplikasi internal adalah proses sistematis untuk mengidentifikasi, mengevaluasi, dan mengatasi kerentanan keamanan dalam aplikasi yang dikembangkan dan digunakan secara internal oleh suatu organisasi. Tujuannya adalah memastikan bahwa aplikasi tidak memiliki celah yang dapat dieksploitasi oleh pihak internal maupun eksternal yang tidak berwenang. Tujuan pengujian *security* aplikasi internal yaitu melindungi data sensitif perusahaan dan pengguna, mencegah akses tidak sah (*unauthorized access*) serta menemukan dan memperbaiki kelemahan sejak dini sebelum dieksploitasi

3. Perbaikan Hasil Pengujian

Perbaikan hasil pengujian merupakan tahap penting untuk menindaklanjuti temuan (*bug*/*error*/ketidaksesuaian) setelah pengujian internal, pre-UAT, dan UAT (*User Acceptance Test*). Tahap ini bertujuan untuk memastikan sistem dapat berfungsi secara optimal sesuai dengan ekspektasi dan kebutuhan pengguna akhir.

Aktivitas perbaikan mencakup:

1. Penelusuran akar masalah (*root cause analysis*) terhadap error atau deviasi hasil pengujian dari spesifikasi sistem.
2. Perbaikan kode, konfigurasi, atau pengaturan sistem yang menyebabkan kegagalan fungsi.
3. Penyesuaian antarmuka, alur proses, atau data model sesuai masukan pengguna.
4. Re-testing atau pengujian ulang setelah perbaikan dilakukan untuk memastikan isu telah terselesaikan.
5. Regression test untuk memastikan perubahan tidak merusak fungsi lain.

4.4 Pengujian *User Acceptance Test* (UAT)

***User Acceptance Test* (UAT)** merupakan bagian penting dari proses validasi sistem, yang dilakukan pada akhir setiap *sprint* atau sebelum *deployment* aplikasi ke *production*. Pengujian ini melibatkan pengguna aplikasi dan perwakilan *stakeholder* untuk memastikan bahwa modul/fitur yang dikembangkan sudah sesuai dengan kebutuhan bisnis dan siap digunakan secara operasional.

Adapun jenis *test* yang dilakukan adalah:

1. Pengujian fungsionalitas bisnis
2. Pengujian *end-to-end*
3. Pengujian integrasi antar modul
4. Perbaikan hasil pengujian

Deliverable dari aktivitas di atas sebagai berikut:

- Dokumen *Checklist User Acceptance Test* (UAT)

4.5 Deployment

Tahapan ini adalah proses mengirimkan (menginstal/menyebarkan) versi aplikasi ke *environment* pengujian atau *server staging*, sebelum akhirnya ke server *production*. Penggunaan *environment staging* memungkinkan tim untuk menguji dan menilai aplikasi dalam *environment* yang mirip dengan *real-user*. Jika aplikasi dapat berjalan secara normal pada *server staging*, aplikasi tersebut akan diterapkan ke server yang sudah ditetapkan (*Staging Server*). Adapun tahapan *deployment* yang dilakukan sebagai berikut:

Aktivitas-aktivitas yang dilakukan pada tahap ini adalah:

1. Menyiapkan *environment staging* dan *production* dan lakukan konfigurasi pada keduanya.
2. Menyiapkan *pipeline* CI/CD untuk otomatisasi proses build, test, dan deploy.
3. Analisis code menggunakan sonarqube sebelum melakukan deploy aplikasi ke *server*.
4. Deploy aplikasi ke *server staging*, kemudian melakukan pengujian di *staging* untuk memastikan aplikasi berjalan normal dan bebas dari *bug*.
5. *Deploy* versi *final* aplikasi ke *server production* dan pastikan seluruh *service* berjalan dengan baik (API, *front-end*, *background jobs*, dll).
6. Memonitor dan melakukan review terhadap jalannya sistem yang baru.
7. Mendukung dan melakukan Pendampingan ke tim teknis IT PT PLN (Persero).

Deliverable dari aktivitas di atas sebagai berikut:

1. Dokumen *Review Code Quality Otomatis SonarQube*
2. Dokumen *Deployment*
3. Dokumen Teknis Pengembangan Perangkat Lunak (DPPL)

4.6 Pemeliharaan dan Dukungan Teknis

Dalam proyek ini *support* yang akan kami berikan adalah:

- Sebagai bagian dari komitmen untuk mendukung fungsi operasional sistem, perusahaan kami telah siap dengan dukungan untuk menangani setiap persoalan yang muncul setelah tahapan produksi selama 3 bulan setelah BAST.
- *Technical Support* dalam melakukan konfigurasi dan implementasi proyek.

Pemeliharaan dan *support system* merupakan kegiatan yang dilakukan setelah selesai tahap go live dengan masa waktu tertentu sesuai perjanjian yang telah disepakati. Hal ini diberikan demi memastikan aplikasi berfungsi optimal tanpa kendala dan bermanfaat bagi operasional PLN.

4.6.1 Dukungan Teknis

Project tim harus memastikan bahwa sistem yang terpasang harus berjalan dengan baik sesuai dengan *system requirement*. Guna mendukung terlaksananya berjalannya sistem (aplikasi dan infrastruktur), maka konsultan melakukan penjadwalan pendampingan dengan tenaga ahli yang dibutuhkan dan waktu berkunjung sesuai waktu kerja setempat.

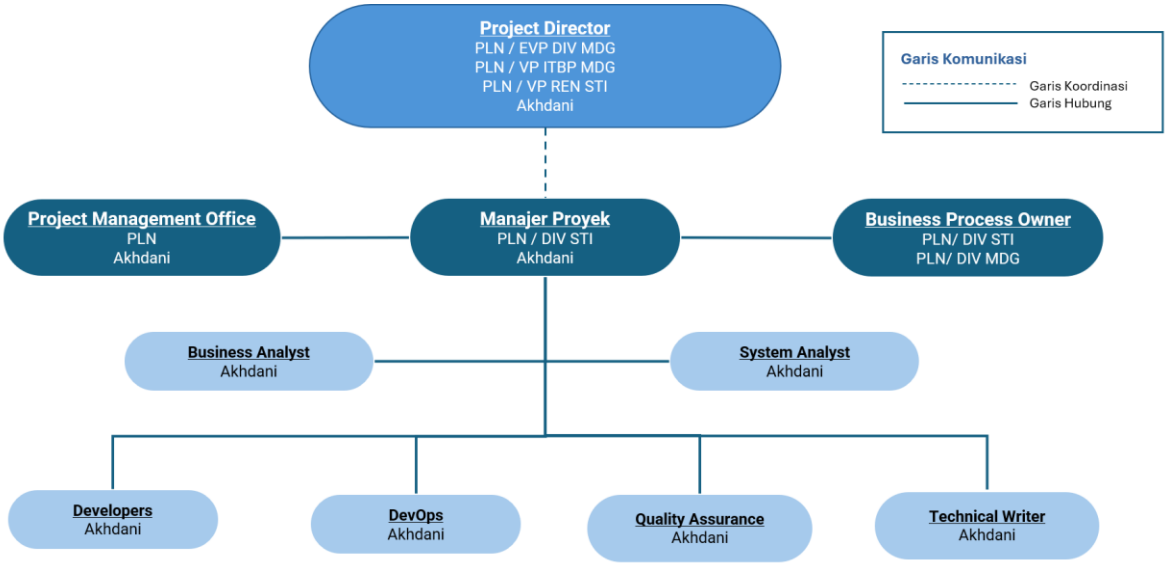
4.6.2 Pendampingan Implementasi

Setelah *go live*, aplikasi di lingkungan PLN akan diimplementasikan oleh user yang terdiri dari user di PLN *Holding*, Divisi MDG, Divisi STI dan Divisi PKK. Akhdani berkomitmen untuk menyediakan tim untuk melakukan pendampingan implementasi aplikasi pada berbagai user tersebut.

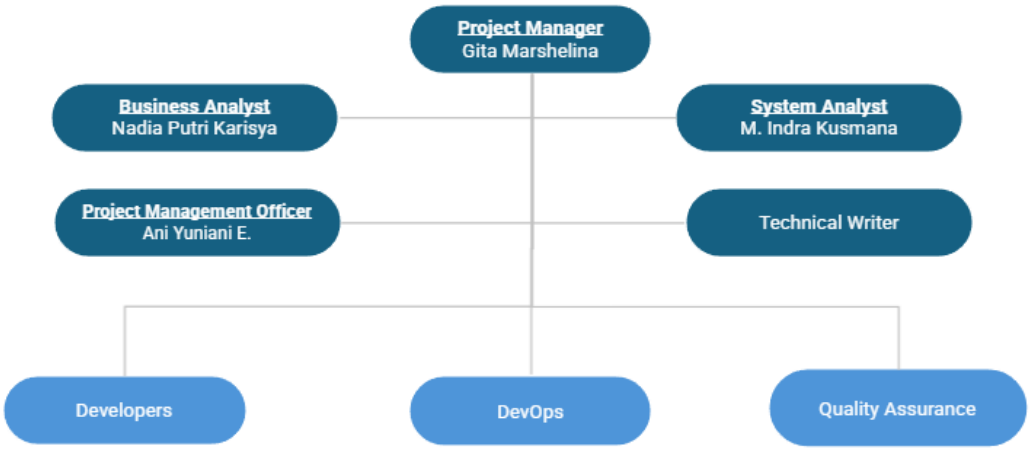
5 ORGANISASI PROYEK BESERTA PERAN DAN TANGGUNG JAWAB TENAGA AHLI

5.1 Struktur Tim Proyek

Untuk mendukung pelaksanaan pekerjaan dan memastikan koordinasi yang efektif antara seluruh pihak yang terlibat, dibentuk struktur organisasi proyek sebagaimana ditunjukkan pada gambar berikut.



Gambar 13 Struktur Organisasi Proyek



Gambar - Struktur Tim

5.2 Tugas dan Tanggung Jawab

Berikut ini tugas dan tanggung jawab dari tiap bagian pelaksana pekerjaan.

1. ***Project Manager***
    - Menyusun rencana kerja proyek (*project plan*) termasuk ruang lingkup, jadwal, anggaran, dan sumber daya.
    - Menentukan metode pelaksanaan, tahapan kerja, serta *milestones* penting proyek.
    - Mengatur dan mengkoordinasikan seluruh anggota tim lintas fungsi (*developer*, *business analyst*, *system analyst*, QA, dan lainnya).
    - Menyusun dan memimpin rapat koordinasi rutin serta memastikan seluruh pihak memahami tanggung jawab masing-masing.
    - Memantau kemajuan setiap tahapan pekerjaan berdasarkan *timeline* dan *deliverables*.
    - Menyampaikan progres proyek, isu yang muncul, dan rekomendasi penyelesaian secara berkala melalui laporan kemajuan.
    - Memastikan hasil kerja tim sesuai dengan standar kualitas yang telah ditetapkan.
2. **Analis Bisnis**
    - Mampu mengidentifikasi, mengumpulkan, dan menganalisis kebutuhan bisnis dari *stakeholder* internal maupun eksternal.
    - Menyusun dokumentasi kebutuhan seperti *Functional Specification*, dan *User Stories*
    - Menterjemahkan kebutuhan bisnis menjadi desain solusi awal yang logis dan teknis.
    - Membuat diagram proses bisnis (BPMN), DFD, ERD, dan *flowchart* sebagai dasar diskusi dengan tim teknis.
    - Berperan sebagai penghubung antara tim bisnis (user, manajemen) dan tim teknis (developer, QA).
    - Membantu tim QA dalam memahami alur bisnis dan merancang skenario pengujian yang realistis.
3. **Analis Sistem**
    - Melakukan analisa terhadap kebutuhan dari sistem yang akan dikembangkan.
    - Berkoordinasi dengan analis bisnis dan *team developer* seperti *programmer.*
    - Menyiapkan dokumen yang dibutuhkan dalam pengembangan.
    - Membantu membuat dokumen pelaporan.
    - Melakukan *control* & *monitoring* terhadap kegiatan yang dilaksanakan.
    - Melakukan pelaporan terhadap jalannya kegiatan.
    - Melakukan perancangan dan desain sistem aplikasi.

-
-
-
- Membangun atau mengembangkan *software* terutama pada tahap requirement, *design* dan sebagian dalam tahap *construction*/ *implementation*.

4. *Developer Senior*

- Memodifikasi *software* yang ada untuk memperbaiki kerusakan dan untuk mengembangkan kinerja sistem.
- Mengembangkan dan mengarahkan pengujian sistem *software* dan prosedur validasi, pemrograman, serta dokumentasi.
- Berkolaborasi dengan analis sistem (*system analyst*), *programmer*, dan pekerja lainnya untuk mendesain sistem & aplikasi, dan untuk memperoleh informasi mengenai limitasi dan kapabilitas proyek, serta persyaratan dari proyek.
- Menganalisis kebutuhan pengguna dan kebutuhan *software* untuk menentukan kelayakan aplikasi.
- Berkoordinasi dengan *programmer junior* dengan memberikan arahan teknis, termasuk pembagian tugas, penentuan prioritas pekerjaan, serta pendampingan dalam proses pengembangan dan penyelesaian fitur aplikasi.

5. *Developer Junior*

- Bekerja sesuai dengan arahan analis sistem dan *developer* senior dalam pembangunan keseluruhan aplikasi
- Mengembangkan program dari grafik alur kerja atau diagram, mengingat kapasitas penyimpanan komputer, kecepatan, dan tujuan penggunaan data *output*.
- Menganalisis, review, dan menulis ulang program, dengan menggunakan grafik dan diagram alur kerja.
- Menulis kode yang memungkinkan *server*, *database*, dan aplikasi berinteraksi dapat bekerja sama untuk melakukan fungsi serta menyampaikan informasi kepada pengguna akhir.
- Berkolaborasi dengan analis sistem (*system analyst*), *programmer*, dan pekerja lainnya untuk mendesain sistem & aplikasi, dan untuk memperoleh informasi mengenai limitasi dan kapabilitas proyek, serta persyaratan dari proyek.
- Memecahkan masalah dan memperbaiki *bug* dalam sistem untuk memastikan server berjalan secara optimal.

- Berpartisipasi dalam semua fase siklus pengembangan, berfokus pada *coding*, pengujian, dan *debugging*.

6. *Quality Assurance*

- Membuat *script test* sesuai dengan kebutuhan aplikasi yang dibuat, sehingga nantinya dapat dijalankan untuk melakukan *automatic testing*
- Meninjau pelaksanaan dan efisiensi kualitas dan inspeksi sistem agar berjalan sesuai rencana, melaksanakan dan memantau pengujian untuk memastikan kualitas sistem aplikasi.
- Merancang sampel prosedur dan petunjuk untuk mencatat dan melaporkan data berkualitas
- Melakukan pengujian pada aplikasi pada aplikasi yang sedang dikembangkan.
- Menganalisis data untuk mengidentifikasi area untuk perbaikan dalam sistem mutu.

7. *Devops Engineer*

- Merancang, membangun, dan mengelola infrastruktur aplikasi baik *on-premise* maupun *cloud*.
- Mengelola dan mengotomasi proses CI/CD untuk *build*, *test*, dan *deployment* aplikasi.
- Mengelola *container* dan *orchestration* (*Docker, Kubernetes*) untuk memastikan skalabilitas aplikasi.
- Melakukan monitoring, *logging*, dan analisis performa sistem serta infrastruktur.
- Menjaga keamanan sistem melalui pengaturan akses, *patching*, dan penerapan *best practice security*.
- Melaksanakan *backup*, *recovery*, dan memastikan *high availability* sistem.
- Menangani *troubleshooting*, *incident response*, serta berkoordinasi dengan tim developer dan QA.

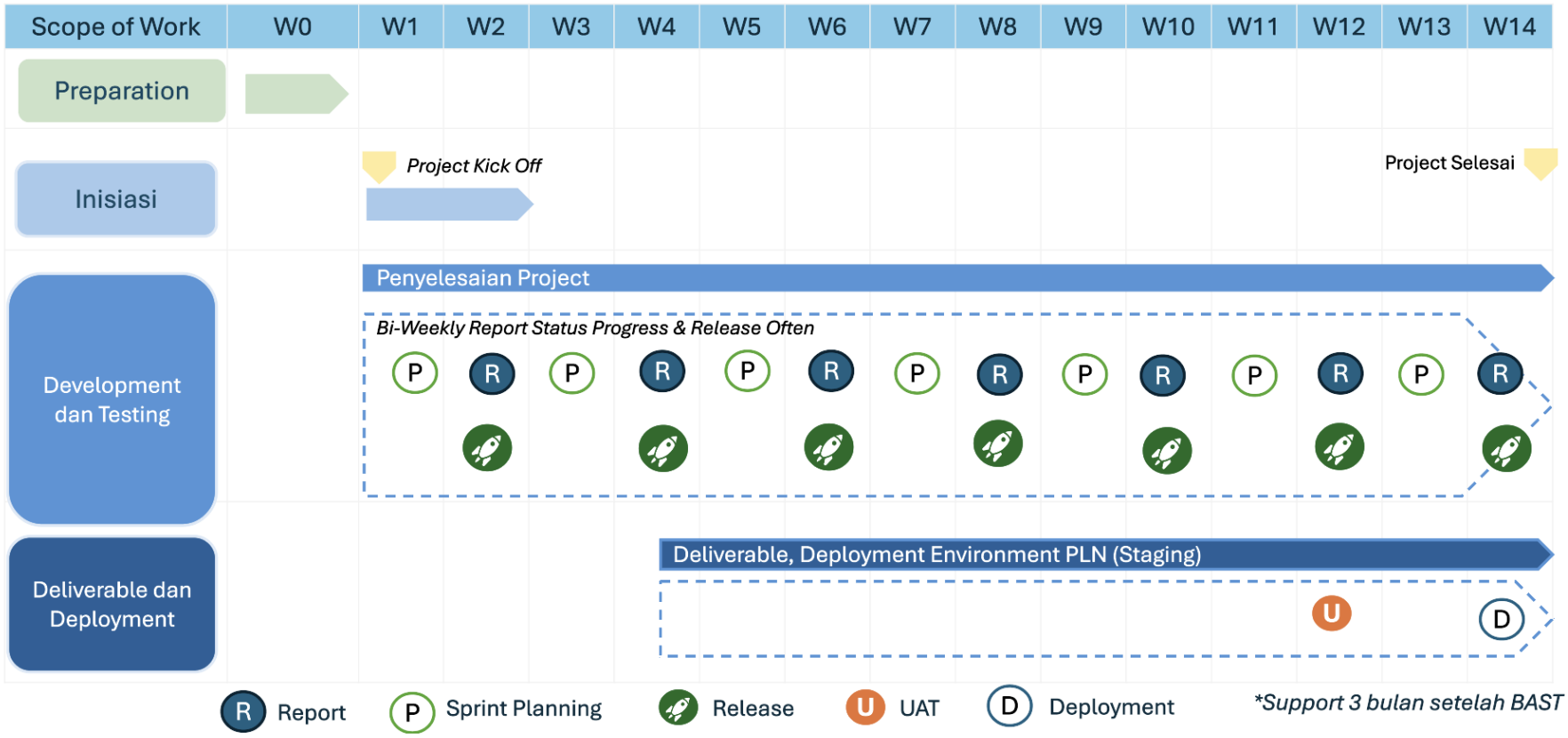
8. *Technical Writer*

- Mendokumentasikan proyek yang disesuaikan dengan standar yang digunakan
- Membantu melakukan pengujian terhadap aplikasi dan pelaporan pekerjaan proyek.
- Menyusun dan menyiapkan dokumentasi *user guide*/manual, *deliverables*, dan lainnya.
- Menangani administrasi proyek.
- Ikut serta dalam rapat proyek, sebagai notulensi & melakukan absensi rapat.

6 TIMELINE

Secara keseluruhan kegiatan **“Pengembangan Aplikasi DIMS”** akan diselesaikan dalam jangka waktu terhitung sejak kontrak ditandatangani. Adapun jadwal pelaksanaan pekerjaan secara umum dapat dilihat pada gambar berikut:

JADWAL PEKERJAAN APLIKASI DIMS



Gambar - Usulan Jadwal Pekerjaan Pengembangan Aplikasi DIMS

7 DELIVERABLE

Adapun rincian *deliverable* yang akan dihasilkan pada pekerjaan **Aplikasi DIMS** akan dijelaskan pada tabel dibawah ini:

No	Produk Laporan	Format	Deskripsi
1.	**Aplikasi hasil pengembangan dan perubahan**		
a.	Aplikasi DIMS	*Source code*	*Fresh copy source Code / final source code* (*free error compile*) hasil pengembangan yang selanjutnya akan menjadi Hak Milik PT PLN (Persero).
b.	*Database* Aplikasi DIMS	*File SQL atau backfup file*	*Database* yang sudah terisi hasil migrasi database dari sistem lama dan database hasil penambahan dan perubahan aplikasi.
2.	**Laporan sepanjang siklus pelaksanaan proyek**		
a.	Laporan Progress (*timesheet*)	*Softcopy* dan *hardcopy*	Laporan yang berisi progres pekerjaan dan jumlah waktu yang dihabiskan dari masing-masing tenaga ahli yang dibuat dan akan dilaporkan berkala setiap bulannya selama pekerjaan berlangsung.
3.	**Dokumen Hasil Pengembangan Teknis**		
a.	Dokumen Pengembangan Perangkat Lunak (DPPL)	*Softcopy* dan *hardcopy*	Merupakan dokumen yang berisi penjelasan spesifikasi kebutuhan sistem, desain sistem dan alur aplikasi, serta penjelasan fitur-fitur hasil penambahan dan perubahan Aplikasi DIMS. Dokumen ini dibuat untuk memudahkan pengembang sebagai referensi untuk pengembangan perangkat lunak berikutnya.
b.	Dokumen Checklist *User Acceptance Test* (UAT)	*Softcopy* dan *hardcopy*	Merupakan laporan hasil pengujian yang menyatakan bahwa setiap item / fitur pada aplikasi telah diuji dan sudah memenuhi kebutuhan penggunanya.

No	Produk Laporan	Format	Deskripsi
c.	Dokumen *Deployment*	*Softcopy* dan *hardcopy*	Merupakan buku petunjuk *deployment* ke *production*, yang difokuskan untuk pembaruan panduan yang sudah ada.
d.	Dokumen Pengujian *Code Review* Menggunakan SonarQube	*Softcopy* dan *hardcopy*	Dokumen hasil analisis kualitas kode aplikasi yang dilakukan secara manual dan rutin menggunakan *Sonar Scanner*
4.	**Dokumen Administrasi**		
a.	Berita Acara Serah Terima Pekerjaan (BAST)	*Hardcopy*	Merupakan dokumen legalitas penyerahan hasil pekerjaan dari konsultan kepada PT PLN (Persero)
b.	Berita Acara Pemeriksaan Pekerjaan (BAPP)	*Hardcopy*	Dokumen yang memuat hasil pemeriksaan dan penilaian terhadap pekerjaan yang telah diselesaikan untuk memastikan kesesuaian dengan ketentuan dan spesifikasi yang disepakati
c.	Berita Acara *User Acceptance Test* (BA UAT)	*Hardcopy*	Dokumen yang mencatat hasil *User Acceptance Test* sebagai bukti bahwa fungsi dan kualitas aplikasi atau sistem telah sesuai dengan kebutuhan

8 PENUTUP

Dengan disampaikannya usulan solusi ini, diharapkan pengembangan sistem yang dilakukan sudah sesuai dengan kebutuhan dan dapat memenuhi harapan pihak PLN sebagai pemilik pekerjaan sehingga dapat mendukung proses pengelolaan anggaran baik di unit maupun pusat, serta dapat menyediakan informasi yang lengkap, cepat dan akurat, membuat sebuah Sistem Informasi yang dapat berjalan dengan baik, mudah digunakan untuk kepentingan pengambilan keputusan.

Demikian kami sampaikan Usulan Solusi untuk **"**Pengembangan Aplikasi *Digital Initiative Management* System (DIMS)" semoga dapat diterima dan digunakan oleh pihak PLN.

Terima Kasih.

Bandung, 11 Juni 2026